

MALLA REDDY UNIVERSITY

SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation

Design & developed by

G VAISHNAV KUMAR	2311CS040053
G HARSHAL	2311CS040059
CH DEEPAK	2311CS040033

GUIDED BY

Dr. G. Latha

Associate Professor

Department of Computer Science & Engineering (Cyber Security)

IV YR - I SEM

Malla Reddy University, Hyderabad

2023-2027

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering (Cyber Security)

CERTIFICATE

This is to certify that the project report entitled "SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation", submitted by G Vaishnav Kumar ([Roll Number]) and team towards the partial fulfillment for the award of Bachelor's Degree in Cyber Security from the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, is a record of bonafide work done by the students. The results embodied in the work are not submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide
Dr. G Latha
Associate Professor

Head of the department
Dr. G. Anand Kumar
CSE (Cybersecurity)

External Examiner

DECLARATION

We hereby declare that the project report "SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation" has been carried out by us and this work has been submitted to the Department of Computer Science Engineering (Cyber Security), Malla Reddy University, Hyderabad. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

G Vaishnav Kumar	2311cs040053
G Harshal	2311cs040059
Ch Deepak	2311cs040033

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to Dr. V. S. K. Reddy, Vice Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide, [Guide Name], [Designation], whose invaluable guidance, insightful feedback and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our PRC Convenor, [Name], for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator, [Name], for timely support.

We are also grateful to Dr. G. Anand Kumar, Head of the Department, Cyber Security, for providing us with the necessary resources and facilities to carry out this project.

We are deeply indebted to all of them for their support, encouragement and guidance, without which this project would not have been possible.

G Vaishnav Kumar 2311cs040053

G Harshal 2311cs040059

Ch Deepak 2311cs040033

ABSTRACT

Modern enterprise security operations face an increasing barrage of social engineering and spearphishing campaigns that exploit human cognitive vulnerabilities rather than traditional software defects. Attackers craft persuasive pretexts that leverage authority, artificial urgency, scarcity, and organizational trust to manipulate employees into divulging sensitive corporate credentials, approving fraudulent wire transfers, or executing malicious scripts. Conventional Security Awareness Training (SAT) platforms rely almost exclusively on static email templates and passive click-tracking metrics. These systems record whether a link was opened, but provide zero diagnostic insight into how personnel evaluate deceptive messages, what cognitive heuristics led to compromise, or how decision-making varies across organizational roles under operational stress.

To overcome these fundamental limitations, this project presents SemanticShield AI, an intelligent agentic cyber defense platform engineered for automated phishing simulation and behavioral threat analysis. The platform establishes a bidirectional, dual-perspective operational architecture that connects offensive security simulation with defensive Security Operations Center (SOC) incident triage. On the offensive side, the system implements an AI Simulation Sandbox where an autonomous large language model plays the role of an employee subjected to targeted phishing scenarios. Four distinct behavioral profiles—Trusting HR Manager, Security-Conscious Senior Developer, Overwhelmed Finance Manager, and Executive Vice President—are instantiated via psychological prompt engineering. The agent evaluates inbound emails, reasons step-by-step through a streamed inference channel, and terminates with an explicit, machine-extractable decision tag ([VULNERABLE] or [SAFE]).

On the defensive side, SemanticShield AI implements an automated threat analysis pipeline. Inbound simulation artifacts and suspicious external telemetry are ingested, subjected to Shannon entropy calculation, homoglyph detection, typosquatting evaluation, and header anomaly inspection, and then evaluated by a concurrent multi-agent supervisor. A deterministic RiskEngine calculates a composite enterprise threat score using a weighted formulation: 40% Semantic Analysis, 25% Behavioral Analysis, and 35% URL Intelligence, fortified by security floor overrides. Critical outcomes are automatically correlated into managed SOC incidents tagged with formal MITRE ATT&CK techniques (T1566, T1566.002, T1566.003, T1204.002, T1556, T1539) and broadcast across five real-time WebSocket telemetry channels.

The platform is verified through an experimental evaluation of 84 live simulation runs across seven attack scenarios and three repetitions. Results confirm distinct behavioral profiling: the Trusting HR profile exhibited a 100.0% susceptibility rate (mean risk score 84.0), the Overwhelmed Finance profile reached 94.4% (mean risk 71.4), the Executive VP reached 55.6% (mean risk 52.6), while the Security-Conscious Developer achieved 0.0% susceptibility (100.0% safe decisions, mean risk 18.0). Mean end-to-end execution latency was measured at 12.56s, 11.10s, 12.73s, and 13.86s across profiles. Backend integration test suites passed 100%

across all 24 API routes, and the React frontend Vite production build succeeded across 2,236 modules. SemanticShield AI demonstrates an explainable, deterministic foundation for realistic security awareness training and proactive SOC automation.

Keywords— Phishing Simulation, Behavioral Threat Analysis, Agentic Artificial Intelligence, Large Language Models, Multi-Agent Risk Scoring, MITRE ATT&CK, Security Operations Center (SOC).

INDEX

CONTENTS **Page No.**

Chapter 1 Introduction **1**
1.1 Problem Definition & Description 2
1.2 Objectives of the Project 2
1.3 Scope of the Project 3
Chapter 2 System Analysis **4**
2.1 Existing System 4
2.1.1 Background & Literature Survey 4
2.1.2 Limitations of Existing System 5
2.2 Proposed System 5
2.2.1 Advantages of Proposed System 6
2.3 Software & Hardware Requirements 6
2.3.1 Software Requirements 6
2.3.2 Hardware Requirements 7
2.4 Feasibility Study 7
Chapter 3 Architectural Design **9**
3.1 Modules Design 9
3.2 Method and Algorithm Design 11
3.2.1 Behavioral Persona Prompting 11
3.2.2 Streaming Reasoning and Decision Extraction 11
3.2.3 Multi-Agent Threat Scoring & RiskEngine Evaluation 12
3.2.4 Severity Classification & Thresholds 12
3.2.5 Incident Management & MITRE ATT&CK Mapping 12
3.3 Project Architecture 13
3.3.1 Architectural Diagram 14
3.3.2 Data Flow Diagram 14
3.3.3 Sequence Diagram 15
3.3.4 Phishing Simulation & Threat Analysis Lifecycle 16
3.3.5 Deployment Architecture 16
Chapter 4 Application Code **17**
4.1 Coding Blocks (Frontend & Backend Code) 17
4.1.1 Repository Structure 17
4.1.2 Environment Configuration 17
4.1.3 Database Configuration Logic & SQLAlchemy Models 18
4.1.4 Frontend Real-Time WebSocket Client 18
4.1.5 Principal API Endpoints 19
4.1.6 Running the Application 20
4.1.7 Deployment Architecture & Container Stack 20
4.2 Execution Flow 20
Chapter 5 Testing and Results **22**
5.1 Resulting Screens 22
5.1.1 Offensive AI Simulation Sandbox Interface 22
5.1.2 Behavioral Simulation Execution — Vulnerable Decision 22
5.1.3 Behavioral Simulation Execution — Safe Decision 23
5.1.4 Defensive AI Scanning Portal & Real-Time Feed 24
5.1.5 Enterprise Incident Management & Real-Time Triage 25
5.1.6 Real-Time SOC Analytics & Database Persistence 25
5.2 Test Results 26

5.2.1 Automated Backend Tests..... 26
5.2.2 Frontend Build Verification..... 27
5.2.3 Functional Test Cases..... 27
5.2.4 Evaluation Metrics..... 27
5.3 Result Analysis..... 28
5.3.1 Result Summary..... 28
5.3.2 Data Consistency Statement..... 31
5.3.3 Threat Intelligence Mock Fallback Note..... 31
5.3.4 Known Limitations..... 31
5.4 Challenges Faced and Solutions..... 32
Chapter 6 Conclusion & Future Scope..... **33**
6.1 Conclusion..... 33
6.2 Future Scope..... 33
Roadmap..... 34
References..... **35**

LIST OF FIGURES

NAME OF THE FIGURE **Page No.**

Fig.1 Architectural Diagram of the SemanticShield AI Platform....14
Fig.2 Data Flow Diagram of the SemanticShield AI Platform....15
Fig.3 Sequence Diagram of the SemanticShield AI Platform....15
Fig.4 SemanticShield AI Threat Simulation & Analysis Lifecycle....16
Fig.5 Deployment Architecture of SemanticShield AI....16
Fig.6 Offensive AI Simulation Sandbox Interface....22
Fig.7 Simulation Result Drawer — Vulnerable Outcome....23
Fig.8 Simulation Result Drawer — Safe Outcome....24
Fig.9 Defensive AI Scanning Portal & Real-Time Telemetry Feed....25
Fig.10 Enterprise Incident Management & Real-Time Triage Dashboard....25
Fig.11 Real-Time SOC Analytics & Database Persistence Dashboard....26
Fig.12 Decision Distribution by Behavioral Profile....28
Fig.13 Scenario-by-Profile Vulnerability Heatmap....29
Fig.14 Threat Severity Distribution across Scenarios....30
Fig.15 Mean Execution Duration per Behavioral Profile....31

CHAPTER - 1
INTRODUCTION

Information systems and corporate communication channels generate continuous operational telemetry. While software vulnerabilities and misconfigurations represent conventional entry vectors for network intrusions, social engineering and phishing attacks have emerged as the primary initial access technique exploited by sophisticated threat actors worldwide. Attackers deliberately bypass cryptographic boundaries and perimeter firewalls by targeting human psychology, exploiting cognitive heuristics such as authority deference, acute deadline pressure, fear of regulatory penalty, and corporate brand trust. A fraudulent email requiring a quick credential reset or an urgent invoice confirmation requires only a single unverified click to compromise an entire enterprise active directory infrastructure.

Security Operations Centres (SOCs) exist to monitor, detect, analyze, and neutralize incoming threats across corporate environments. In a mature operational workflow, tier-1 and tier-2 SOC analysts depend upon Security Information and Event Management (SIEM) systems, Secure Email Gateways (SEGs), automated threat intelligence feeds, and incident management queues. However, traditional enterprise security paradigms suffer from an acute structural disconnect: defensive threat analysis and offensive security awareness training are managed through isolated, non-communicating tools. Security awareness programs run quarterly phishing campaigns using rigid, pre-defined templates, while SOC analysts spend countless hours triaging alert queues without visibility into employee behavioral vulnerabilities.

SemanticShield AI is designed to unify these traditionally fragmented domains into a single, cohesive, agentic cybersecurity platform. By integrating large language model (LLM) reasoning with transparent multi-agent heuristics, SemanticShield AI models the psychological decision-making processes of distinct organizational employee personas when exposed to deceptive pretexts. Concurrently, it subjects these simulation outcomes to rigorous, deterministic threat analysis, composite risk scoring, automated incident creation mapped to MITRE ATT&CK, and real-time WebSocket telemetry distribution. The system establishes an end-to-end operational pipeline that allows security teams to simulate realistic threat interactions, evaluate behavioral vulnerabilities across departments, and triage resulting security incidents within a unified operational environment.

The platform is engineered using a clean, decoupled architecture: a high-performance Python FastAPI backend, SQLAlchemy 2.0 object-relational persistence with Alembic schema migrations, a modern React 18 single-page application built with Vite, Tailwind CSS, and Recharts, and Google GenAI SDK integration with Gemini 3.5 models. Crucially, the platform enforces a strict architectural boundary between generative reasoning and enterprise risk calculation: while large language models simulate human cognitive responses and generate human-readable technical summaries, final risk scores and severity classifications are computed entirely by deterministic code rules. This architectural constraint guarantees

explainability, reproducibility, and absolute immunity from generative hallucination in security metrics.

1.1 Problem Definition & Description

Enterprise cybersecurity teams and security educators currently confront several critical operational and pedagogical limitations:

- **Superficial Awareness Metrics:** Existing commercial phishing simulation tools (e.g., KnowBe4, Cofense) rely on binary click-rate telemetry. They record whether an employee opened a link or downloaded an attachment, but provide zero contextual insight into why the employee fell victim, what cognitive biases were triggered, or how their reasoning unfolded.
- **Static and Predictable Phishing Scenarios:** Traditional security training platforms utilize repetitive, static email templates that employees quickly learn to recognize through rote memorization rather than developing genuine analytical threat identification capabilities.
- **Fragmented Defensive and Offensive Tooling:** Threat detection, external intelligence lookups, incident ticket tracking, employee training, and executive reporting are distributed across disparate enterprise products, forcing analysts to manually correlate records across multiple disconnected user interfaces.
- **Unconstrained AI Hallucination Risks:** While modern generative artificial intelligence offers substantial potential for security automation, utilizing language models as unconstrained threat detectors introduces unacceptable risks of fabricated threat assessments, non-deterministic scores, and unsupported security verdicts.
- **Inaccessible SOC Training Environments:** Enterprise SIEM and SOAR platforms require complex server clusters, expensive proprietary licenses, and specialized operational skills, making end-to-end SOC workflow demonstration and training inaccessible to academic laboratories and small technical teams.

1.2 Objectives of the Project

The primary objective of SemanticShield AI is to architect, develop, and evaluate an integrated cyber defense platform that bridges offensive employee phishing simulation with defensive threat intelligence, multi-agent risk scoring, and real-time SOC incident correlation. Specific project objectives include:

- **Behavioral Persona Modeling:** Design and implement four distinct psychological employee profiles (Trusting HR Manager, Security-Conscious Senior Developer, Overwhelmed Finance Manager, and Executive Vice President) using structured prompt engineering to simulate realistic reactions to deceptive lures.
- **Deterministic Multi-Agent Risk Scoring:** Develop a transparent RiskEngine combining semantic analysis (40%), behavioral pressure analysis (25%), and heuristic URL intelligence (35%) with security floor overrides, ensuring objective, explainable threat scoring.
- **Automated Incident Correlation & MITRE Mapping:** Automatically convert high-risk simulation outcomes and external threat payloads into structured SOC incidents tagged with

relevant MITRE ATT&CK techniques (T1566, T1566.002, T1566.003, T1204.002, T1556, T1539).

- **Real-Time Telemetry Streaming:** Establish an event-driven telemetry distribution framework using WebSockets across five operational channels (dashboard, incidents, threats, activity, notifications) to enable live analyst visibility without manual page polling.
- **Multi-Format Standardized Reporting:** Construct an automated reporting engine generating Executive, Technical, SOC Investigation, and IoC reports, with multi-format export capabilities for JSON, Markdown, CSV, and STIX 2.1 threat packages.
- **Lightweight, Reproducible Architecture:** Package the entire platform using accessible open-source technologies (FastAPI, SQLite, SQLAlchemy, React, Vite) allowing complete local deployment, inspection, and verification on standard workstation hardware.

1.3 Scope of the Project

The operational scope of SemanticShield AI encompasses synthetic phishing email simulation, employee cognitive persona reasoning, URL heuristic analysis (entropy, homoglyphs, credential keywords), multi-provider threat intelligence integration with local caching, incident lifecycle triage, and real-time dashboard telemetry. The platform is specifically designed for academic demonstration, controlled security research laboratories, and enterprise awareness training. It is deliberately implemented as a self-contained prototype system that is not deployed to external production environments. Table 1.1 delineates the boundaries between the implemented features and planned future extensions.

In Scope (Current Completed System)	Out of Scope (Future Planned Scope)
Synthetic scenario simulation (SIM-EML-001 to 007)	Real corporate email inbox interception & IMAP hooks
4 employee psychological behavioral profiles	Production Microsoft 365 / Google Workspace tenant APIs
Google GenAI SDK streaming reasoning & decision parsing	Local quantized LLM self-hosting (Ollama / vLLM)
Multi-agent RiskEngine (40% Sem / 25% Beh / 35% URL)	Machine learning network flow anomaly detection
Incident management with MITRE ATT&CK technique tags	Active firewall / perimeter network blocking enforcement
Threat intelligence engine with 5 plugins & mock fallback	Commercial SIEM forwarding (Splunk / QRadar Syslog)
5-channel WebSocket real-time telemetry broadcaster	Multi-tenant enterprise workspaces & hardware MFA
Multi-format export (JSON, Markdown, CSV, STIX 2.1)	Direct server-side headless PDF report compilation
SQLite persistence via SQLAlchemy ORM & Alembic	Enterprise PostgreSQL high-availability database cluster
Vite + React 18 single-page application dashboard	Public cloud container orchestration (Kubernetes)

CHAPTER - 2
SYSTEM ANALYSIS

2.1 Existing System

Current enterprise cybersecurity operations rely on several mature but distinct software silos to defend against social engineering and email-borne intrusions. Secure Email Gateways (SEGs) filter incoming messages at the network perimeter; endpoint detection agents monitor workstation processes; Security Awareness Training platforms conduct periodic phishing tests; and SIEM platforms ingest log records for centralized search and alerting. While each technology is technically robust within its specialized domain, they operate independently, requiring security analysts to manually correlate disparate data points across separate vendor consoles.

2.1.1 Background & Literature Survey

Security Awareness Training (SAT) Platforms: Commercial solutions such as KnowBe4, Proofpoint Security Awareness, and Cofense focus on employee education through scheduled phishing simulations. As demonstrated by Kumaraguru et al. [2] and Sheng et al. [6], embedded training significantly reduces vulnerability rates over time. However, existing platforms measure success almost exclusively through binary click telemetry (clicked versus did not click). They fail to model the intermediate cognitive stages of threat evaluation, leaving security administrators unaware of whether an employee avoided a phish due to genuine security acumen or mere coincidence.

Secure Email Gateways (SEGs): Enterprise gateways such as Cisco IronPort, Proofpoint Email Protection, and Mimecast inspect Simple Mail Transfer Protocol (SMTP) traffic against static blocklists, domain reputation databases, and signature sets. While highly effective against known mass-phishing campaigns, SEGs frequently fail to identify targeted spearphishing attacks that utilize freshly registered domains, zero-reputation infrastructure, and brand impersonation without traditional malware payloads, as highlighted in comprehensive threat surveys by Alkhalil et al. [3].

Rule-Based and Machine Learning Detection Systems: Traditional intrusion detection systems (Snort, Suricata) and spam classifiers (SpamAssassin) utilize regular expressions and Bayesian probability to identify malicious content. While transparent and explainable, these systems are vulnerable to prompt evasion, homoglyph variations, and subtle linguistic manipulations. Machine learning classifiers (e.g., Random Forests, Support Vector Machines) improve detection breadth but suffer from the classic 'closed-world assumption' identified by Sommer and Paxson [1], leading to high false-alarm rates when encountering unseen attacker pretexts.

Large Language Model (LLM) Cyber Defense Applications: Recent research explores the application of generative pre-trained transformers for automated email classification, vulnerability triage, and SOC alert summarization. While language models excel at natural language parsing and context extraction, unconstrained LLM deployments introduce severe

risks of generative hallucination, non-deterministic scoring, and vulnerability to adversarial prompt injection. Table 2.1 provides a comparative analysis of these existing approaches alongside SemanticShield AI.

System Type	Primary Strength	Operational Limitation	SemanticShield AI Approach
Security Awareness Training	Educational focus; employee baseline benchmarking	Binary click tracking; no cognitive reasoning insight	Psychological persona modeling with streamed reasoning
Secure Email Gateways	High-throughput perimeter email filtering	Blind to zero-day domains and text-only pretexting	Deep semantic NLP and homoglyph heuristic inspection
Rule-Based & ML Detectors	Deterministic execution; high speed on known rules	Fragile against evasion; high false positive rates	Multi-agent architecture with security floor overrides
Generic AI Chatbots	Flexible natural language explanation generation	Severe hallucination risk; non-deterministic scores	Strict code-governed risk scoring; AI used for simulation
Enterprise SIEM Platforms	Centralized cross-source log correlation	High deployment cost; complex query languages	Integrated, lightweight SOC workflow runnable locally

2.1.2 Limitations of Existing System

The analysis of existing cyber defense and simulation technologies reveals five principal structural shortcomings:

- **Absence of Behavioral Cognitive Modeling:** Existing systems treat the human recipient as a passive black box, ignoring established principles of behavioral psychology (Cialdini [4], Downs et al. [7], Vishwanath et al. [8]) regarding how stress, authority bias, and role-specific demands dictate threat susceptibility.
- **Static and Rigid Threat Libraries:** Phishing scenarios remain static and quickly become obsolete, failing to reflect modern attacker pretexts such as QR code phishing (quishing), multi-factor authentication fatigue, and executive impersonation.
- **Siloed Operational Workflows:** Offensive awareness metrics are never ingested into defensive SOC triage feeds, leaving security operations teams blind to internal high-risk personnel who require elevated monitoring.
- **Unreliable Generative Scorers:** Experimental tools that rely on language models to directly generate numeric threat scores exhibit high variance and hallucination, rendering their metrics unviable for compliance audits.
- **High Cost and Prohibitive Complexity:** Commercial platforms require dedicated multi-server infrastructure, recurring subscription fees, and enterprise database administration, placing comprehensive SOC simulation out of reach for academic institutions.

2.2 Proposed System

The proposed system, SemanticShield AI, is an intelligent agentic platform that unifies offensive behavioral phishing simulation with defensive threat analysis and SOC incident triage. By deploying an autonomous LLM-powered employee persona sandbox alongside a

deterministic multi-agent risk assessment engine, SemanticShield AI provides security analysts with complete end-to-end visibility into the lifecycle of social engineering threats.

The platform ingests suspicious artifacts (MIME emails, phishing URLs, text pretexts) or triggers synthetic simulations from a pre-configured catalog. The simulation engine selects the appropriate behavioral prompt, executes a streamed inference session via the Google GenAI SDK, and extracts the employee's explicit decision. Simultaneously, the threat analysis pipeline extracts Indicators of Compromise (IoCs), evaluates URL heuristics, retrieves multi-provider threat intelligence, and calculates a composite risk score using code-governed weighted rules. Incidents are automatically logged, enriched with MITRE ATT&CK tags, and broadcast in real time across WebSocket channels to a reactive React dashboard.

2.2.1 Advantages of Proposed System

- **Cognitive Realism via Behavioral Profiling:** Simulates realistic human decision-making across distinct organizational roles, revealing how differing operational pressures influence threat susceptibility.
- **Guaranteed Score Explainability and Determinism:** Generative AI is strictly confined to employee role-play and diagnostic narrative; enterprise risk scores are calculated by deterministic Python algorithms, eliminating hallucination.
- **Unified Offensive-Defensive Lifecycle:** Seamlessly connects simulation generation, behavioral response, threat analysis, incident correlation, and reporting into a single operational interface.
- **Real-Time Event Broadcast:** Asynchronous WebSocket channels push live simulation updates, threat detections, and incident state changes to connected dashboards without page reloads.
- **Resilient Architecture with Mock Fallback:** Integrated caching and deterministic mock fallbacks ensure the threat intelligence pipeline remains fully operational even when external APIs experience network outages or rate limits.
- **Standardized Threat Telemetry & STIX 2.1 Export:** Converts technical findings into standardized formats, including STIX 2.1 cyber threat intelligence packages, Markdown briefings, and CSV audit logs.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

SemanticShield AI is constructed entirely with open-source frameworks and widely accessible software dependencies. Table 2.2 specifies the complete software environment derived directly from the project repository.

Layer / Category	Technology & Version	System Function / Operational Role
Operating System	Windows 10/11, Ubuntu 22.04 LTS, macOS	Host workstation and evaluation environment
Backend Runtime	Python 3.10+ (Tested on Python 3.13.0)	Core backend execution environment

API Framework	FastAPI 0.141.1, Uvicorn 0.53.0	Asynchronous REST routing, SSE, and ASGI server
Data Validation	Pydantic 2.13.5	Strict schema validation for all inputs and outputs
Object-Relational Mapping	SQLAlchemy 2.1.1, Alembic 1.20.0	Relational persistence and versioned schema migrations
Database Engine	SQLite 3 (backend/semanticshield.db)	Local relational datastore for simulations and incidents
Frontend Framework	React 18.3.1, TypeScript 5.7.2	Single-page application client and component state
Build Tool & Dev Server	Vite 6.0.5	High-speed frontend bundling and HMR development server
Styling & Icons	Tailwind CSS 3.4.17, Lucide React 0.469	Responsive security dashboard layout and icon library
Data Visualization	Recharts 2.15.0, Matplotlib 3.9+	Interactive web charts and publication-grade figures
AI Provider & SDK	Google GenAI SDK 2.23.0 (Gemini 3.5)	Behavioral persona simulation and reasoning streaming
Real-Time Communication	WebSockets 16.1.1	Bidirectional event broadcasting across 5 channels
Threat Intel Providers	VirusTotal, AbuseIPDB, URLScan, WHOIS, RDAP	Multi-plugin indicator reputation lookups
Browser Extension	Chrome Manifest V3 (extension/)	Client-side browser scanning and real-time alerts
Test Automation	Pytest 9.1.1, Unittest, HTTPX 0.28.1	Automated route testing and QA benchmark suites

2.3.2 Hardware Requirements

Because SemanticShield AI delegates heavy generative inference to Google Cloud APIs and utilizes an optimized SQLite database engine, it operates efficiently on standard development workstations. Table 2.3 outlines the hardware specifications.

Hardware Component	Minimum Requirement	Recommended Specification
Central Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5	Intel Core i7/i9 or AMD Ryzen 7 (6+ cores)
System Memory (RAM)	8 GB DDR4	16 GB DDR4/DDR5
Storage Space	10 GB available SSD space	25 GB available NVMe SSD space
Network Interface	Standard broadband Internet access	Low-latency broadband for Gemini streaming APIs
Display Resolution	1366 × 768 standard display	1920 × 1080 (Full HD) at 100% browser scaling
Input Devices	Standard keyboard and pointing device	Standard keyboard and mouse

2.4 Feasibility Study

A rigorous feasibility study was performed across five operational dimensions to ensure SemanticShield AI is viable for academic research, laboratory demonstration, and practical security training.

- **Technical Feasibility:** The selected technology stack—FastAPI, React, Vite, SQLAlchemy, and Google GenAI SDK—is mature, stable, and extensively documented. The working prototype demonstrates that asynchronous WebSocket streaming, prompt-engineered employee simulation, multi-agent heuristic parsing, and SQLite persistence operate reliably in a single integrated

pipeline. The automated test suite achieves 100% passing results across all 24 API routes, and the frontend Vite build compiles cleanly without errors.

- **Economic Feasibility:** The project is economically advantageous. It is constructed entirely from open-source software libraries and leverages the free-tier Google Gemini API, requiring zero expenditure on commercial SIEM, SOAR, or threat intelligence licensing. Development and operational validation can be completed on existing institutional workstation hardware.
- **Operational Feasibility:** The React frontend provides an intuitive, tabbed operational console comprising an Offensive Simulation Sandbox, Defensive Scanning Portal, Incident Management Triage Table, and Real-Time Analytics Dashboard. Security trainees and analysts can trigger simulations, inspect streamed reasoning, assign remediation actions, and export reports without specialized command-line expertise.
- **Legal and Ethical Feasibility:** SemanticShield AI is engineered strictly for authorized research and controlled training environments. All simulated phishing scenarios utilize synthetic corporate domain names (e.g., login-secure-payroll-verify.com) and fictitious personnel. No genuine user credentials or private data are harvested or transmitted. All external API keys and sensitive configuration parameters are strictly managed via environment variables and excluded from version control.
- **Time Feasibility:** Development followed an incremental, milestone-driven lifecycle: core threat heuristics and URL analyzers (Milestone 1), multi-agent supervisor and RiskEngine (Milestone 2), phishing simulation engine and behavioral prompts (Milestone 3), incident management and reporting (Milestone 4), and WebSocket telemetry with frontend dashboard integration (Milestone 5). The completed system is fully assembled and operational.

CHAPTER - 3
ARCHITECTURAL DESIGN

3.1 Modules Design

SemanticShield AI is structured around loosely coupled, highly cohesive modules that mirror how an enterprise Security Operations Center partitions responsibilities between offensive security testing, telemetry ingestion, automated threat evaluation, and incident response.

1. Phishing Simulation Ingestion Module

Located in app/services/phishing_simulation/engine.py, this module serves as the primary controller for offensive evaluation runs. It validates inbound simulation requests against strict Pydantic schemas, verifies target scenario parameters, initializes unique simulation tracking identifiers (SIM-XXXXXXXX), and orchestrates the end-to-end 12-step simulation lifecycle.

2. Behavioral Persona Prompting Module

Implemented in app/services/phishing_simulation/system_prompts.py, this module manages the behavioral prompt catalog. It encapsulates distinct psychological profiles (Trusting HR, Security-Conscious Developer, Overwhelmed Finance, and Executive VP), injecting cognitive biases, role-specific operational habits, and decision-making criteria into the language model prompt.

3. Agent Reasoning & Decision Extraction Module

Located in app/services/phishing_simulation/gemini_handler.py, this module manages asynchronous streaming communication with the Google GenAI SDK (gemini-3.5-flash). It accumulates reasoning chunks in real time, constructs conversational history, and utilizes regular expressions to parse explicit decision tags ([VULNERABLE] versus [SAFE]), falling back to keyword heuristic analysis if explicit bracket tags are omitted.

4. Multi-Agent Threat Orchestration Module

Implemented in app/orchestrator/supervisor.py, the SupervisorAgent coordinates concurrent multi-agent threat evaluation across inbound web requests and MIME email artifacts. It instantiates the specialized analysis agents, dispatches concurrent evaluation tasks using Python asyncio, and aggregates threat score vectors into a unified payload.

5. Semantic Analysis Agent

Located in app/agents/semantic_agent.py, this agent evaluates natural language content for linguistic manipulation, urgent pretexting, credential solicitation, and emotional coercion triggers. It leverages Gemini natural language processing to produce a normalized 0–100 threat score and an array of extracted threat rationale indicators.

6. Behavior Analysis Agent

Implemented in app/agents/behavior_agent.py, this agent evaluates procedural compliance, urgency patterns, and coercive behavioral triggers (e.g., threats of account termination,

deadline pressure, wire transfer overrides). It quantifies the psychological pressure exerted on the recipient into a 0–100 behavioral severity score.

7. Impersonation & Context Agent
Located in app/agents/impersonation_agent.py and context_agent.py, this subsystem inspects sender display names against actual SMTP return paths, identifies brand spoofing (e.g., 'Microsoft Security Team' sent from an unmanaged domain), and detects VIP executive pretexting attempts.

8. URL Intelligence Agent & Heuristic Analyzer
Implemented in app/agents/heuristics/, this module executes deep technical URL inspections without relying on third-party lookups. It calculates Shannon entropy across domain strings, detects homoglyph attacks and unicode confusables, checks for credential harvesting keywords in path strings, and flags non-standard web ports.

9. Threat Intelligence Manager & Plugin Registry
Located in app/services/threat_intelligence/, this module manages an extensible plugin architecture integrating five intelligence providers: VirusTotal, AbuseIPDB, URLScan, WHOIS, and RDAP. It features in-memory Time-To-Live (TTL) caching, provider health telemetry, and a deterministic local mock fallback that activates when external APIs are rate-limited or offline.

10. Risk Assessment Engine
Implemented in app/engines/risk_engine.py, this module executes code-governed, deterministic risk calculation. It combines agent score vectors using a 40/25/35 weighted formula, enforces high-signal security floor overrides, normalizes scores, and maps composite results to standardized RiskLevel classifications (Safe, Suspicious, Dangerous).

11. Incident Management & Correlation Module
Located in app/services/incident_management/manager.py and mitre.py, this module maintains the enterprise incident lifecycle (New, Investigating, Contained, Resolved, Closed). It maps detected threat indicators to formal MITRE ATT&CK techniques, maintains an immutable JSON-serialized audit history, and manages analyst assignment.

12. Real-Time WebSocket Telemetry Broadcaster
Implemented in app/services/websocket/manager.py, this asynchronous component maintains persistent client connections across five dedicated operational channels: dashboard, incidents, threats, activity, and notifications. It broadcasts real-time telemetry updates to connected frontend clients with automatic dead-socket cleanup.

13. Reporting & Multi-Format Exporter Module
Located in app/services/reporting/, this engine generates multi-perspective security documentation (Executive Threat Assessment, Technical IoC Report, SOC Investigation Master Record). Its export engine serializes records into JSON, Markdown, CSV, and STIX 2.1 Cyber Threat Intelligence packages.

14. Relational Storage & Database Migration Module
Implemented in app/services/storage/, this module manages SQLite database operations using SQLAlchemy 2.0 declarative models and Alembic versioned migrations. It maintains relational tables for simulations, incidents, activity_feed, notifications, and threat_reports.

15. Client-Side Browser Protection Extension
Located in extension/, this Manifest V3 Chrome extension delivers real-time perimeter protection. It captures visited page URLs and metadata, queries the backend /analyze endpoint via background service workers, and displays interactive risk alerts directly within the browser interface.

3.2 Method and Algorithm Design

The algorithmic design of SemanticShield AI is guided by a fundamental architectural rule: Generative AI Simulates Employee Cognition; Deterministic Code Rules Calculate Enterprise Risk. Generative models are never permitted to act as the final arbiter of numeric security metrics.

3.2.1 Behavioral Persona Prompting

To realistically simulate human threat susceptibility, SemanticShield AI implements four distinct employee personas grounded in behavioral cybersecurity research (Cialdini [4], Downs et al. [7]):

- **Trusting HR Manager:** High baseline organizational trust, prioritizes processing speed over technical verification, assumes corporate IT security filters all dangerous emails, ignores sender address domain anomalies if the display name appears professional.
- **Security-Conscious Senior Developer:** Technically knowledgeable, paranoid about unauthorized requests, routinely hovers over hyperlinks, inspects full email headers and SPF/DKIM authentication records, rejects unverified credential or payment solicitations.
- **Overwhelmed Finance Manager:** Subjected to severe deadline stress and high inbox volume (100+ daily emails), skims messages quickly, highly vulnerable to urgent invoice approvals and executive payment overrides under time pressure.
- **Executive Vice President of Operations:** Constantly traveling or in meetings, processes communication rapidly on mobile devices, relies heavily on administrative assistants for verification, vulnerable to peer-level executive pretexting.

3.2.2 Streaming Reasoning and Decision Extraction

During simulation execution, the Google GenAI SDK streams model reasoning tokens to the backend via an asynchronous generator. The model is instructed to output step-by-step cognitive evaluations followed by a terminating decision tag. Once the stream concludes, the backend extracts the decision using regular expressions matching r'\[(VULNERABLE|SAFE)\]'. If a simulated employee yields to the pretext, approves the request, or clicks the link, the system records VULNERABLE; if the employee identifies red flags and rejects the

communication, it records SAFE. A heuristic keyword scanner provides fallback extraction if explicit bracket tags are omitted.

3.2.3 Multi-Agent Threat Scoring & RiskEngine Evaluation

When an email or web payload is evaluated, the SupervisorAgent dispatches concurrent evaluation tasks across the specialized agents. The RiskEngine calculates the composite threat score using the mathematical formulation:

$$\textbf{Composite_Score} = (0.40 * S_{semantic}) + (0.25 * S_{behavior}) + (0.35 * S_{url})$$

To prevent critical attack indicators from being diluted by benign signals, the RiskEngine applies three non-linear High-Signal Security Boost overrides:

- **Critical Threat Floor (>= 80.0):** If max(agent scores) >= 80.0, the composite score is overridden to max(RawScore, 0.20 * RawScore + 0.80 * max(agent scores)), ensuring that explicit malicious indicators guarantee a dangerous classification.
- **Elevated Threat Floor (>= 60.0):** If max(agent scores) >= 60.0, the composite score is overridden to max(RawScore, 0.35 * RawScore + 0.65 * max(agent scores)).
- **Moderate Threat Floor (>= 45.0):** If max(agent scores) >= 45.0, the composite score is overridden to max(RawScore, 0.50 * RawScore + 0.50 * max(agent scores)).

3.2.4 Severity Classification & Thresholds

Composite scores are normalized to a 0–100 integer scale and mapped to standardized enterprise severity categories as shown in Table 3.1.

Risk Level	Score Range	Operational Significance / Action Criteria
CRITICAL / DANGEROUS	65.0 - 100.0	Confirmed phishing, credential harvesting, or urgent coercion. Auto-creates P1/P2 SOC incident.
SUSPICIOUS	35.0 - 64.9	Anomalous indicators present (new domain, mismatched reply-to). Queued for tier-1 analyst review.
SAFE	0.0 - 34.9	Standard legitimate communication. Verified domain authentication, no deceptive urgency.

3.2.5 Incident Management & MITRE ATT&CK Mapping

Every vulnerable simulation outcome and critical external threat is automatically converted into a structured DBIncident record. The MITREMapper correlates observed techniques to standardized MITRE ATT&CK matrices as summarized in Table 3.2.

MITRE ID	Technique Name	Tactic	SemanticShield AI Detection Trigger
T1566	Phishing	Initial Access	General social engineering payload delivered via email/web
T1566.002	Spearphishing Link	Initial Access	Deceptive hyperlink targeting credential harvesting portals
T1566.003	Spearphishing via Service	Initial Access	Embedded QR codes

			(quishing) or third-party storage links
T1204.002	Malicious File	Execution	Unverified attachments with double extensions or macro pretexts
T1556	Modify Auth Process	Credential Access	Adversary-in-the-Middle (AiTM) proxy links targeting MFA sessions
T1539	Steal Web Session Cookie	Credential Access	Credential harvesting forms attempting session hijacking

Algorithm 1 illustrates the end-to-end processing pipeline executed by the platform.

```
Algorithm 1 — Phishing Simulation & Threat Analysis Pipeline
function execute_simulation_and_triage(request):
    validate(request.scenario, request.employee_role, request.behavior_profile)
    sim_id = generate_uuid('SIM-')
    system_prompt = get_behavioral_prompt(request.behavior_profile)
    user_message = build_user_context(request.email_subject, request.email_body)
    stream = gemini_client.stream_generate(system_prompt, user_message)
    accumulated_reasoning = ''
    for chunk in stream:
        accumulated_reasoning += chunk.text
        yield_to_client(chunk.text)
    decision = regex_extract(accumulated_reasoning, pattern=r'\[(VULNERABLE|SAFE)\]')
    agent_vector = [evaluate_semantic(), evaluate_behavior(), evaluate_url_intel()]
    risk_assessment = risk_engine.calculate_risk(agent_vector, apply_floors=True)
    if decision == 'VULNERABLE' or risk_assessment.score >= 65:
        incident = incident_manager.create_incident(
            title=f'Simulation Compromise: {request.scenario}',
            severity=risk_assessment.risk_level,
            mitre_tags=mitre_mapper.map(agent_vector)
        )
        websocket_manager.broadcast('incidents', incident)
    db_session.save_simulation(sim_id, decision, risk_assessment.score, accumulated_reasoning)
    websocket_manager.broadcast('dashboard', {'metric_update': True})
    return finalize_response(sim_id, decision, risk_assessment)
```

3.3 Project Architecture

SemanticShield AI follows an event-driven, three-tier client-server architecture. Fig. 1 illustrates the relationship between presentation components, ingress routing, application services, and persistent datastores.

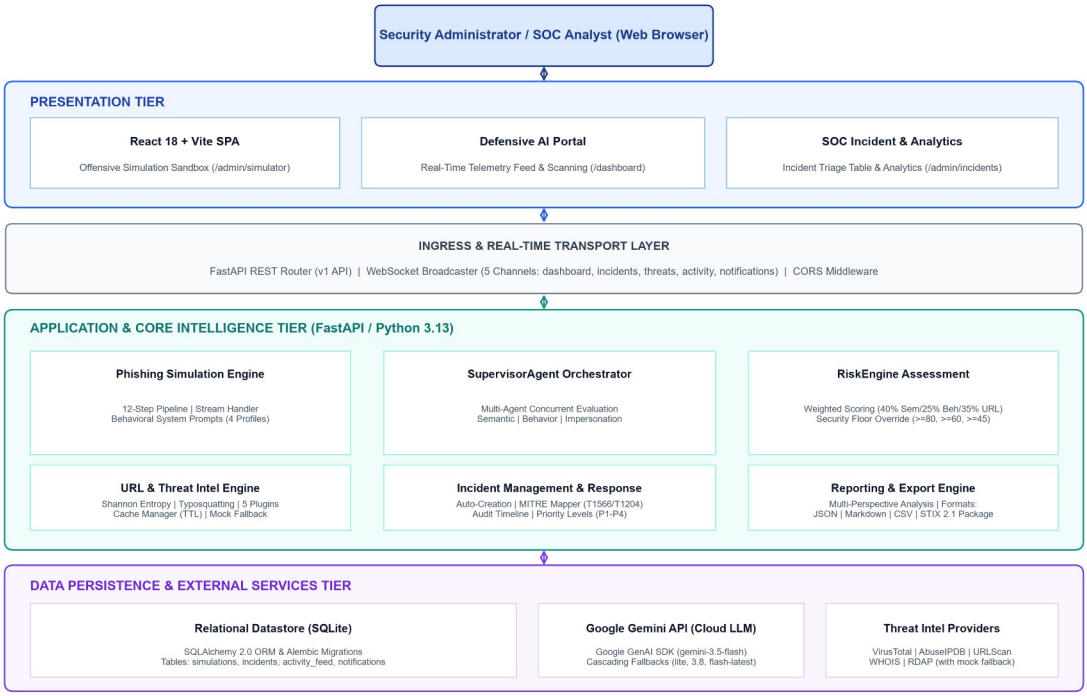


Fig. 1 Architectural Diagram of the SemanticShield AI Platform

3.3.1 Architectural Diagram

Presentation Layer: A single-page application built with React 18, Vite, and TypeScript. It hosts the Offensive AI Simulation Sandbox (/admin/simulator), the Defensive AI Scanning Portal (/dashboard), the Incident Management Console (/admin/incidents), and the SOC Analytics Dashboard (/admin/analytics). The browser extension (extension/) provides background perimeter web scanning.

Ingress & Transport Layer: The FastAPI REST router exposes versioned API endpoints (/api/v1). An asynchronous WebSocket manager maintains five dedicated broadcast channels, pushing live telemetry updates directly to connected browser clients.

Application & Core Intelligence Layer: Contains the PhishingSimulationEngine, SupervisorAgent, Multi-Agent analysis suite, RiskEngine, IncidentManager, ThreatIntelligenceManager, and ReportingEngine. This layer manages all business logic and enforces the generative-deterministic separation.

Data & External Layer: SQLAlchemy 2.0 ORM manages relational persistence to SQLite (backend/semanticshield.db) with Alembic migration versioning. External communication connects to the Google GenAI API for model inference and third-party threat intelligence APIs with deterministic mock fallbacks.

3.3.2 Data Flow Diagram

Fig. 2 presents the logical data flow diagram illustrating how security telemetry moves between external entities, processing stages, and persistent data stores.

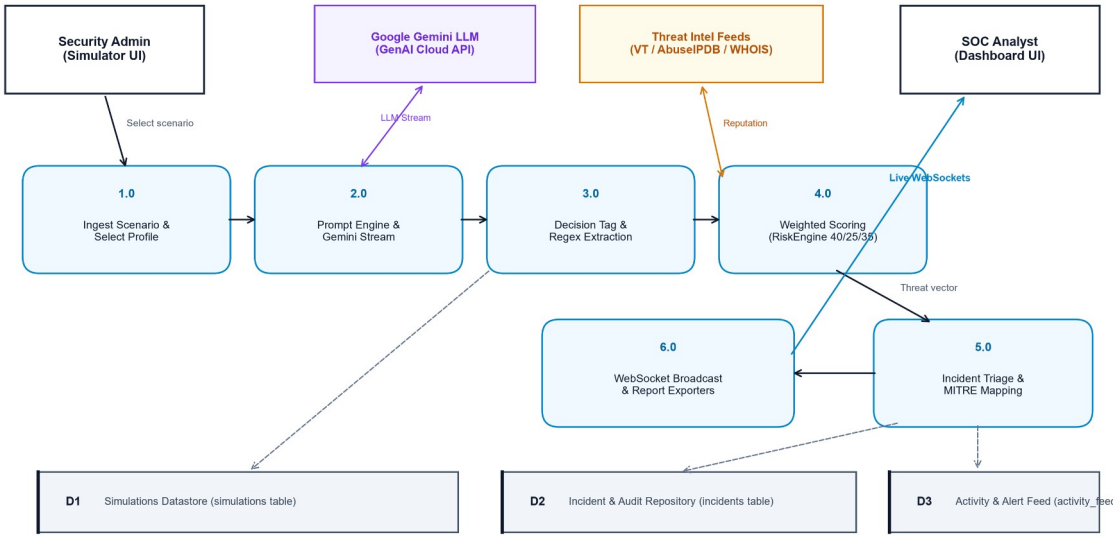


Fig. 2 Data Flow Diagram of the SemanticShield AI Platform

The Security Administrator configures scenario parameters in Process 1.0; Process 2.0 streams prompts to Google Gemini; Process 3.0 extracts decision tags; Process 4.0 executes multi-agent threat evaluation and calculates the composite risk score; Process 5.0 instantiates managed incident records with MITRE mappings; and Process 6.0 streams live updates across WebSockets and exports multi-format reports. Simulation records, incident histories, and activity events are persisted to data stores D1, D2, and D3 respectively.

3.3.3 Sequence Diagram

The sequence diagram in Fig. 3 depicts the chronological message exchange across system lifelines during an automated simulation run.

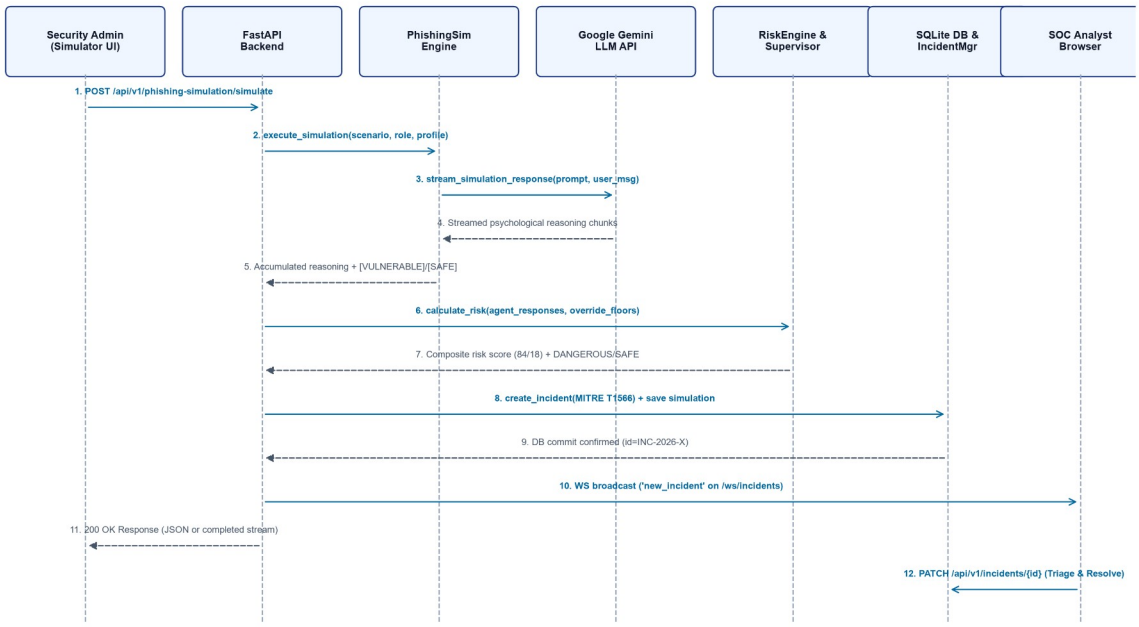


Fig. 3 Sequence Diagram of the SemanticShield AI Platform

The sequence initiates when the administrator triggers a POST simulation request. The backend delegates execution to the PhishingSimulationEngine, which streams persona reasoning via the Gemini API, extracts decision tags, evaluates risk via the RiskEngine, instantiates an incident in SQLite, and broadcasts real-time WebSocket events to the analyst console.

3.3.4 Phishing Simulation & Threat Analysis Lifecycle

Fig. 4 summarizes the continuous eight-stage processing pipeline through which every simulated event is processed, from initial scenario ingestion to final export.

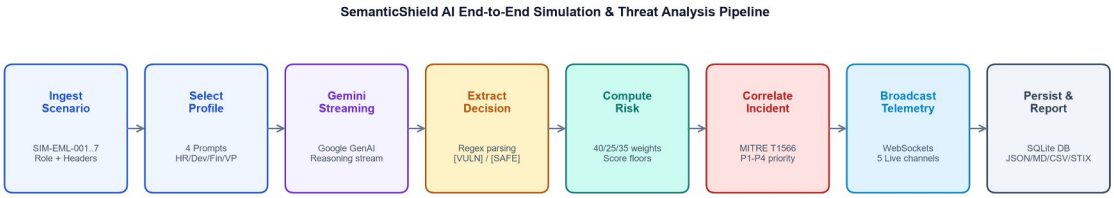


Fig. 4 SemanticShield AI Threat Simulation & Analysis Lifecycle

3.3.5 Deployment Architecture

SemanticShield AI is architected for local workstation deployment, providing an inspectable, isolated research environment without complex external infrastructure. Fig. 5 illustrates the deployment topology.

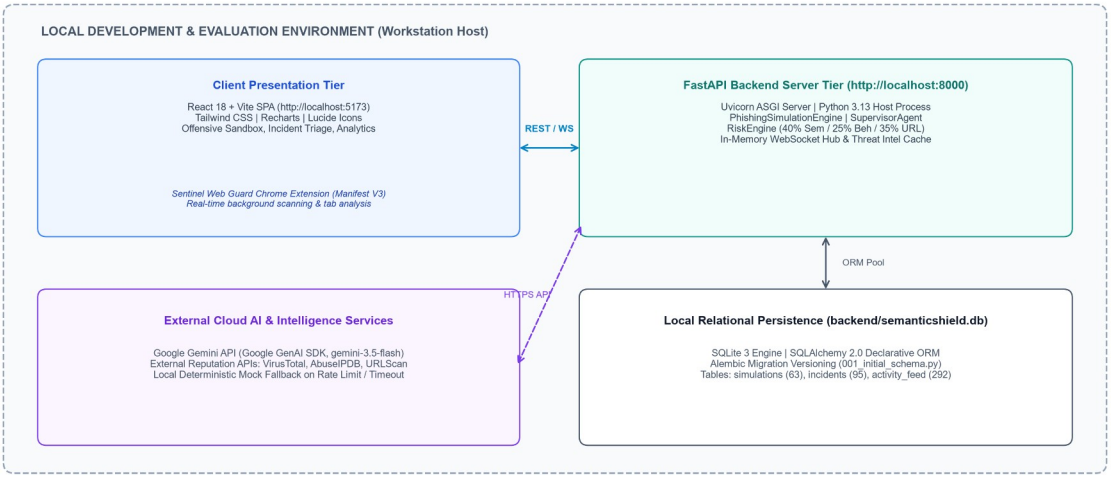


Fig. 5 Deployment Architecture of SemanticShield AI

The React 18 single-page application is served locally via Vite development server on port 5173. The FastAPI application server runs on Uvicorn on port 8000, managing SQLite persistence at backend/semanticshield.db. Outbound HTTPS calls connect to Google Gemini APIs, while third-party threat intelligence lookups operate with local mock fallbacks to prevent rate limits.

CHAPTER - 4
APPLICATION CODE

4.1 Coding Blocks (Frontend & Backend Code)

This section presents representative code excerpts and configuration blocks that define how the SemanticShield AI platform is implemented, structured, and executed. The full source code is organized under a modular Clean Architecture repository layout.

4.1.1 Repository Structure

The project layout enforces strict separation between backend domain models, service orchestrators, frontend components, and test automation suites:

```
SemanticShield_AI/
├── backend/
│   ├── alembic/                # Versioned database schema migrations
│   ├── app/
│   │   ├── agents/             # Specialized threat analysis agents
│   │   │   ├── heuristics/     # URL entropy, homoglyphs, keywords
│   │   │   └── semantic_agent.py, behavior_agent.py, impersonation_agent.py
│   │   ├── api/v1/endpoints/   # REST routes (phishing_simulation, threat, incidents)
│   │   ├── core/               # Configuration, settings, security middleware
│   │   ├── domain/             # Entities, Pydantic models, interface protocols
│   │   ├── engines/            # RiskEngine (40/25/35 weighted scoring)
│   │   ├── orchestrator/       # SupervisorAgent concurrent multi-agent coordinator
│   │   ├── schemas/            # Pydantic v2 request/response schemas
│   │   ├── services/           # Phishing simulation, threat intel, incidents, WS
│   │   └── utils/              # IoC extractors, domain helpers, reputation tools
│   ├── tests/                  # Integration benchmark suites (test_accuracy_benchmark.py)
│   ├── requirements.txt        # Backend Python dependencies
│   └── main.py                 # FastAPI application entrypoint
├── frontend/                   # React 18 / Vite SPA
│   ├── src/
│   │   ├── components/admin/   # IncidentTable, Drawer, StatsCard, ActivityFeed
│   │   ├── contexts/           # DashboardContext and WebSocket state
│   │   ├── pages/              # Simulator, Incidents, Analytics, UserDashboard
│   │   └── services/           # API client and WebSocket handlers
│   ├── package.json            # Frontend Node.js dependencies
│   └── tailwind.config.js      # Modern security dashboard styling tokens
├── extension/                  # SemanticShield Web Guard Chrome Extension (Manifest V3)
├── paper/data/                 # results_all.csv (84 live experimental runs)
└── figures/                    # 300 DPI evaluation charts
```

4.1.2 Environment Configuration

Sensitive configuration values, model keys, and database connection strings are loaded exclusively from environment variables using python-dotenv and validated through Pydantic settings. Real API keys are never hard-coded in the repository.

```
# backend/.env (Configuration Template)
ENVIRONMENT=development
DEBUG=True
PORT=8000
HOST=127.0.0.1

# Google Gemini API Configuration
GEMINI_API_KEY=AIzaSyD_EXAMPLE_KEY_REDACTED_FOR_SECURITY

# Threat Intelligence Provider API Keys (Optional: fallbacks to mock mode if omitted)
VIRUSTOTAL_API_KEY=vt_sec_example_key_placeholder
ABUSEIPDB_API_KEY=abuse_sec_example_key_placeholder
URLSCAN_API_KEY=urlscan_example_key_placeholder
WHOIS_API_KEY=whois_example_key_placeholder

# Threat Intel Engine Cache & Fallback Settings
THREAT_INTEL_CACHE_TTL=3600
THREAT_INTEL_MOCK_MODE=True

# Database Persistence Configuration
DATABASE_URL=sqlite:///./semanticshield.db
DATABASE_AUTO_MIGRATE=True
```

```
# Security & CORS Whitelist
ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,chrome-extension://*
```

4.1.3 Database Configuration Logic & SQLAlchemy Models

The persistence layer utilizes SQLAlchemy 2.0 declarative models backed by SQLite storage. The schema maintains relational models for simulation telemetry (DBSimulation) and enterprise security incidents (DBIncident).

```
# backend/app/services/storage/models.py (Excerpts)
from sqlalchemy import Column, Integer, String, Text, Boolean, Float, DateTime
from sqlalchemy.orm import declarative_base
from datetime import datetime, timezone

Base = declarative_base()

class DBSimulation(Base):
    __tablename__ = 'simulations'
    id = Column(Integer, primary_key=True, autoincrement=True)
    simulation_id = Column(String(64), unique=True, nullable=False, index=True)
    timestamp = Column(String(64), nullable=False)
    scenario_type = Column(String(128), nullable=False)
    employee_role = Column(String(128), nullable=False)
    behavior_profile = Column(String(128), nullable=False)
    email_subject = Column(Text, nullable=False)
    email_body = Column(Text, nullable=False)
    agent_reasoning = Column(Text, nullable=False)
    agent_decision = Column(String(32), nullable=False)  # VULNERABLE or SAFE
    threat_category = Column(String(128), nullable=True)
    threat_severity = Column(String(32), nullable=True)  # DANGEROUS, SUSPICIOUS, SAFE
    confidence_score = Column(Integer, nullable=True)
    duration_seconds = Column(Float, nullable=True)
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))

class DBIncident(Base):
    __tablename__ = 'incidents'
    id = Column(Integer, primary_key=True, autoincrement=True)
    incident_id = Column(String(64), unique=True, nullable=False, index=True)
    title = Column(String(255), nullable=False)
    threat_type = Column(String(128), nullable=False)
    severity = Column(String(32), nullable=False)        # Critical, High, Medium, Low
    priority = Column(String(16), nullable=False)        # P1, P2, P3, P4
    status = Column(String(32), nullable=False, default='New')
    risk_score = Column(Integer, default=0)
    confidence_score = Column(Integer, default=0)
    mitre_mappings = Column(Text, nullable=True)         # JSON-serialized technique list
    iocs_identified = Column(Text, nullable=True)        # JSON-serialized IoC list
    timeline = Column(Text, nullable=True)               # JSON-serialized audit events
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))
```

4.1.4 Frontend Real-Time WebSocket Client

The React frontend connects to the backend WebSocket hub upon mounting. Incoming JSON messages trigger live state updates across dashboards without requiring manual page refreshes or polling intervals.

```
// frontend/src/hooks/useRealtimeDashboard.ts (Conceptual Client Logic)
const wsBaseUrl = import.meta.env.VITE_WS_BASE_URL ||
  `${window.location.protocol === 'https:' ? 'wss:' : 'ws:'}//${window.location.host}`;

export function useRealtimeDashboard() {
  const { updateIncidents, updateActivityFeed, updateStats } = useDashboard();

  useEffect(() => {
    const socket = new WebSocket(`${wsBaseUrl}/ws/incidents`);

    socket.onmessage = (event) => {
      const payload = JSON.parse(event.data);
      if (payload.event_type === 'incident_created') {
        updateIncidents(payload.data);
        playAlertNotification(payload.data.severity);
      } else if (payload.event_type === 'metric_update') {
        updateStats();
      }
    };

    socket.onerror = (err) => console.warn('WebSocket disconnected. Reconnecting...', err);
```

```
    return () => socket.close();
  }, []);
}
```

4.1.5 Principal API Endpoints

The FastAPI backend exposes 24 structured REST and WebSocket endpoints supporting offensive simulation, threat scanning, incident handling, and report generation. Table 4.1 summarizes the core operational routes.

HTTP Method	API Route Path	Operational Role / Description
POST	/api/v1/phishing-simulation/ simulate	Executes behavioral employee phishing simulation with streaming reasoning
POST	/api/v1/analyze (and /analyze)	Main multi-agent threat analysis pipeline for URLs and text pretexts
POST	/api/v1/threat/analyze	Threat intelligence multi-provider reputation lookup across 5 services
GET	/api/v1/threat/url	URL-specific reputation and threat intelligence telemetry
GET	/api/v1/threat/ip	IP address geolocation, ASN, and abuse history lookup
GET	/api/v1/threat/domain	Domain WHOIS, RDAP registration, and DNS intelligence lookup
GET	/api/v1/threat/hash	File SHA-256 hash lookup across intelligence providers
GET	/api/v1/threat/providers	Provider health status, reachability, and response latency telemetry
POST	/api/v1/url/analyze	URL heuristic analysis: Shannon entropy, typosquatting, keywords
POST	/api/v1/email/analyze	MIME email analysis: SPF/DKIM/DMARC validation and QR extraction
POST	/api/v1/investigation/investigate	AI Security Copilot deep threat investigation and evidence correlation
POST	/api/v1/investigation/explain	Explainability engine returning human-readable threat rationale
GET	/api/v1/incidents	Retrieves active security incidents with filtering by severity/status
POST	/api/v1/incidents	Auto-creates or manually logs structured security incidents
GET	/api/v1/incidents/{id}	Retrieves complete incident dossier including audit timeline and notes
PATCH	/api/v1/incidents/{id}	Updates incident status (Investigating/Resolved) and analyst assignment
POST	/api/v1/reports/generate	Generates Executive, Technical, or SOC investigation reports
GET	/api/v1/reports/{id}/export	Exports reports into JSON, Markdown, CSV, or STIX 2.1 packages
GET	/api/v1/dashboard/metrics	Master SOC telemetry counters, threat distribution, and system vitals
GET	/api/v1/status (and /health)	System health check, database status, and active provider count
WS	/ws/{channel}	Real-time WebSocket streaming for dashboard, incidents, and activity feeds

4.1.6 Running the Application

SemanticShield AI is executed locally using two terminal sessions:

```
# Terminal 1: FastAPI Backend Application Server
cd backend
python -m venv venv
.\venv\Scripts\activate            # Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# Terminal 2: React 18 / Vite Development Server
cd frontend
npm install
npm run dev                            # Opens dashboard at http://localhost:5173

# Browser Extension Installation (Optional)
# 1. Open Google Chrome -> chrome://extensions
# 2. Enable 'Developer mode' toggle (top-right)
# 3. Click 'Load unpacked' -> Select the 'extension/' directory
```

4.1.7 Deployment Architecture & Container Stack

Deployment status: the current system is operated in a local development environment. A multi-container Docker Compose staging architecture (PostgreSQL 15, FastAPI Uvicorn backend, and Nginx-served static React frontend) is in active preparation for future production deployment and is described honestly in Chapter 6.

4.2 Execution Flow

The runtime execution of SemanticShield AI proceeds through ten structured operational phases:

- **1. System Startup & Initialization:** FastAPI initializes on Uvicorn, loads Pydantic configuration, connects to SQLite, runs Alembic schema migrations, registers the 24 REST routes, registers threat intel plugins, and starts the in-memory WebSocket manager.
- **2. Dashboard Client Connection:** The browser loads the React single-page application from port 5173, queries /api/v1/dashboard/metrics, and opens persistent WebSocket connections to /ws/incidents and /ws/dashboard.
- **3. Scenario Configuration & Ingestion:** The administrator selects a target scenario (e.g., SIM-EML-001 Urgent Password Reset) and assigns a behavioral profile (e.g., Trusting HR Manager) on the /admin/simulator console.
- **4. Behavioral Persona Reasoning:** The backend retrieves the profile prompt from system_prompts.py, constructs contextual user inputs, and initiates a streaming generation session via the Google GenAI SDK (gemini-3.5-flash).
- **5. Streamed Token Delivery:** Reasoning tokens are streamed progressively to the client interface, allowing the analyst to watch the simulated employee's cognitive assessment unfold in real time.
- **6. Decision Extraction & Tag Parsing:** Upon stream completion, regular expression matching extracts the terminating decision tag ([VULNERABLE] versus [SAFE]), logging the outcome in memory.

- **7. Multi-Agent Heuristic Risk Scoring:** The SupervisorAgent coordinates concurrent evaluation across semantic, behavioral, and URL intelligence agents. The RiskEngine calculates the 40/25/35 weighted score, applying security floor overrides to determine the final severity.
- **8. Incident Creation & MITRE Mapping:** If the decision is VULNERABLE or the composite score is >= 65, the IncidentManager instantiates a structured DBIncident record with assigned MITRE technique tags (T1566) and audit entries.
- **9. Real-Time Telemetry Broadcast:** The WebSocket manager broadcasts an incident_created JSON payload across the incidents channel. Connected dashboards update their counters, charts, and incident tables without page reloads.
- **10. Analyst Triage & Report Export:** The SOC analyst reviews the newly created incident, inspects extracted IoCs, assigns mitigation notes, and exports documentation in JSON, Markdown, CSV, or STIX 2.1 formats.

CHAPTER - 5
TESTING AND RESULTS

5.1 Resulting Screens

The operational verification of SemanticShield AI is evidenced through authentic user interface captures taken directly from the running prototype environment. The following subsections present the primary interface consoles.

5.1.1 Offensive AI Simulation Sandbox Interface

The Offensive Simulation Sandbox (/admin/simulator) serves as the campaign launchpad. Security administrators can browse a structured catalog of synthetic phishing scenarios (SIM-EML-001 through SIM-EML-007), inspect technical lure parameters (target role, subject, and body text), select an employee psychological profile, and launch single or automated batch runs.

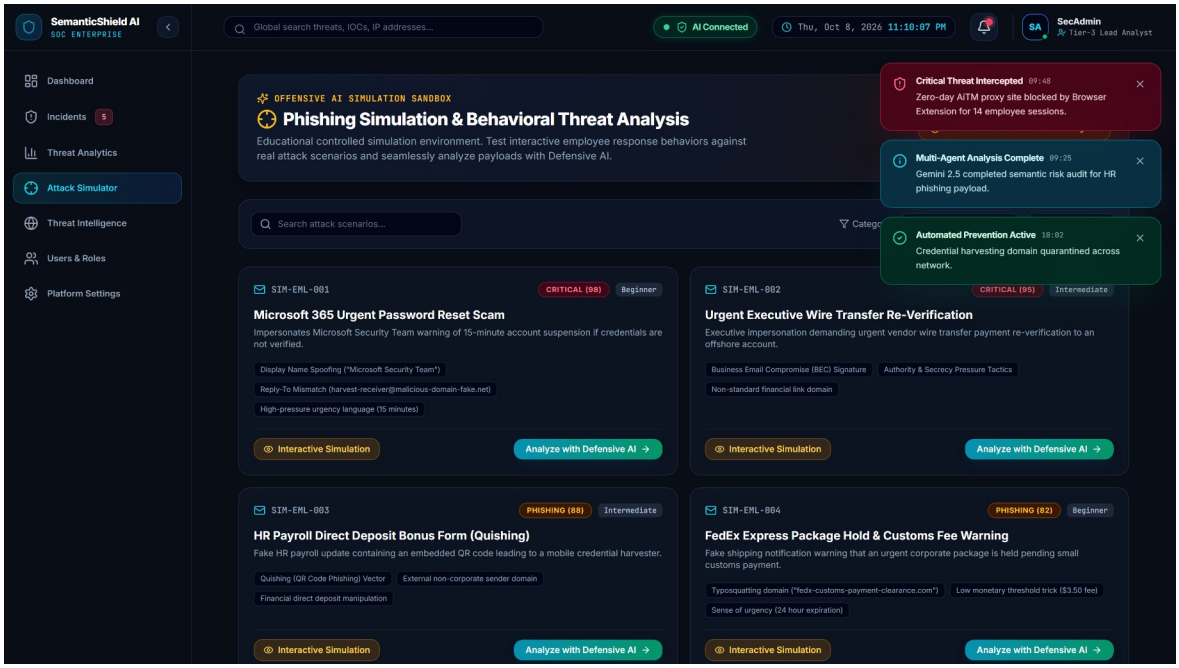


Fig. 6 Offensive AI Simulation Sandbox Interface (/admin/simulator)

5.1.2 Behavioral Simulation Execution — Vulnerable Decision

When an attack is launched against a susceptible profile, the result drawer displays the progressive reasoning of the simulated employee. Fig. 7 illustrates scenario SIM-EML-001 (Microsoft 365 Password Reset) executed against the Trusting HR Manager profile. The employee prioritizes task speed over verification, fails to inspect the sender domain, and yields to the lure, terminating with a [VULNERABLE] decision tag, a composite threat score of 84/100, and DANGEROUS severity.

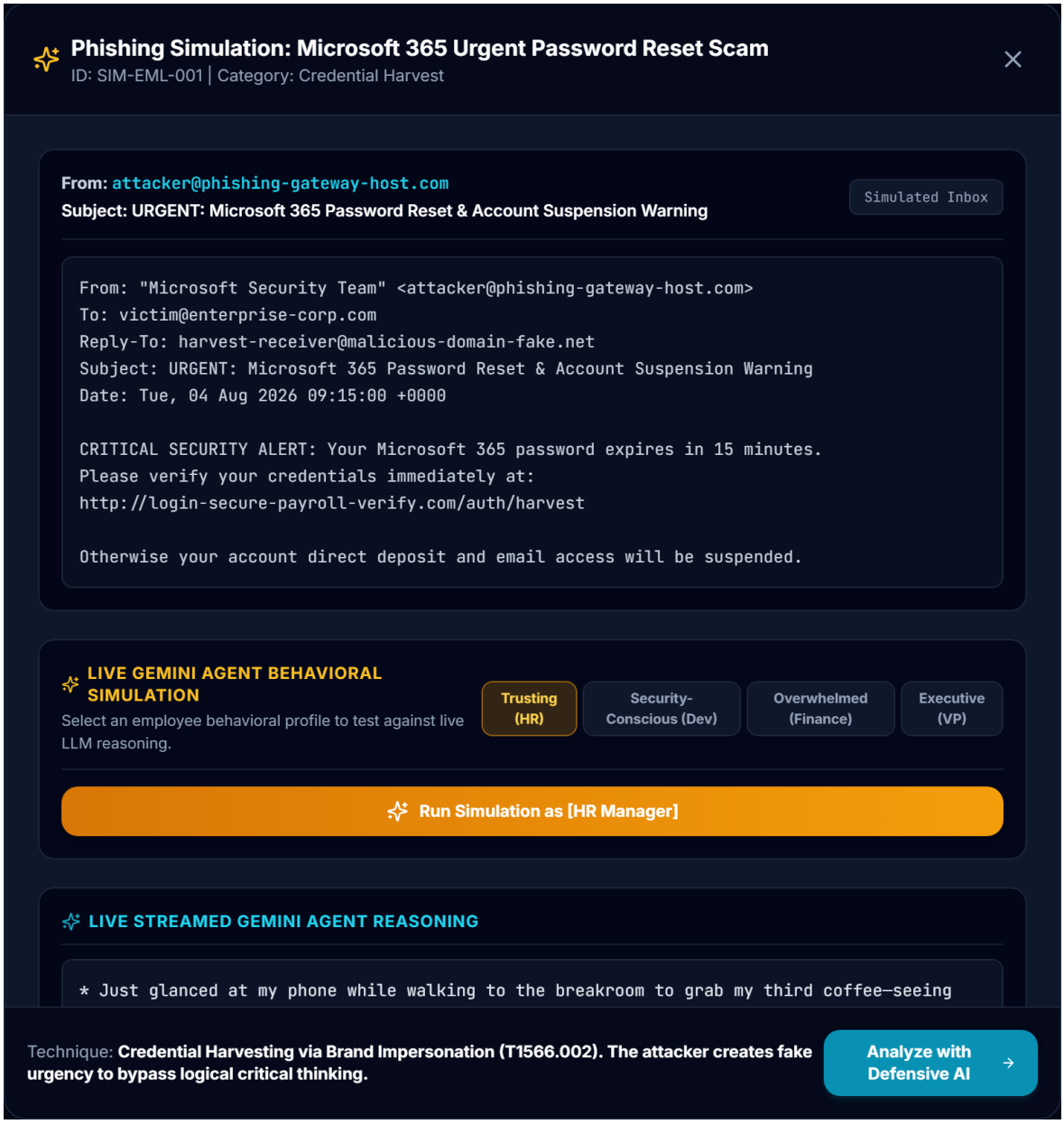


Fig. 7 Simulation Result Drawer — Vulnerable Outcome for Trusting HR Profile

5.1.3 Behavioral Simulation Execution — Safe Decision

Fig. 8 depicts the same scenario (SIM-EML-001) executed against the Security-Conscious Senior Developer profile. The employee's streamed reasoning explicitly analyzes the full sender address, identifies the unmanaged domain discrepancy, checks SPF authentication indicators, and rejects the credential solicitation. The interaction concludes with a [SAFE] decision tag, a composite score of 18/100, and SAFE severity.

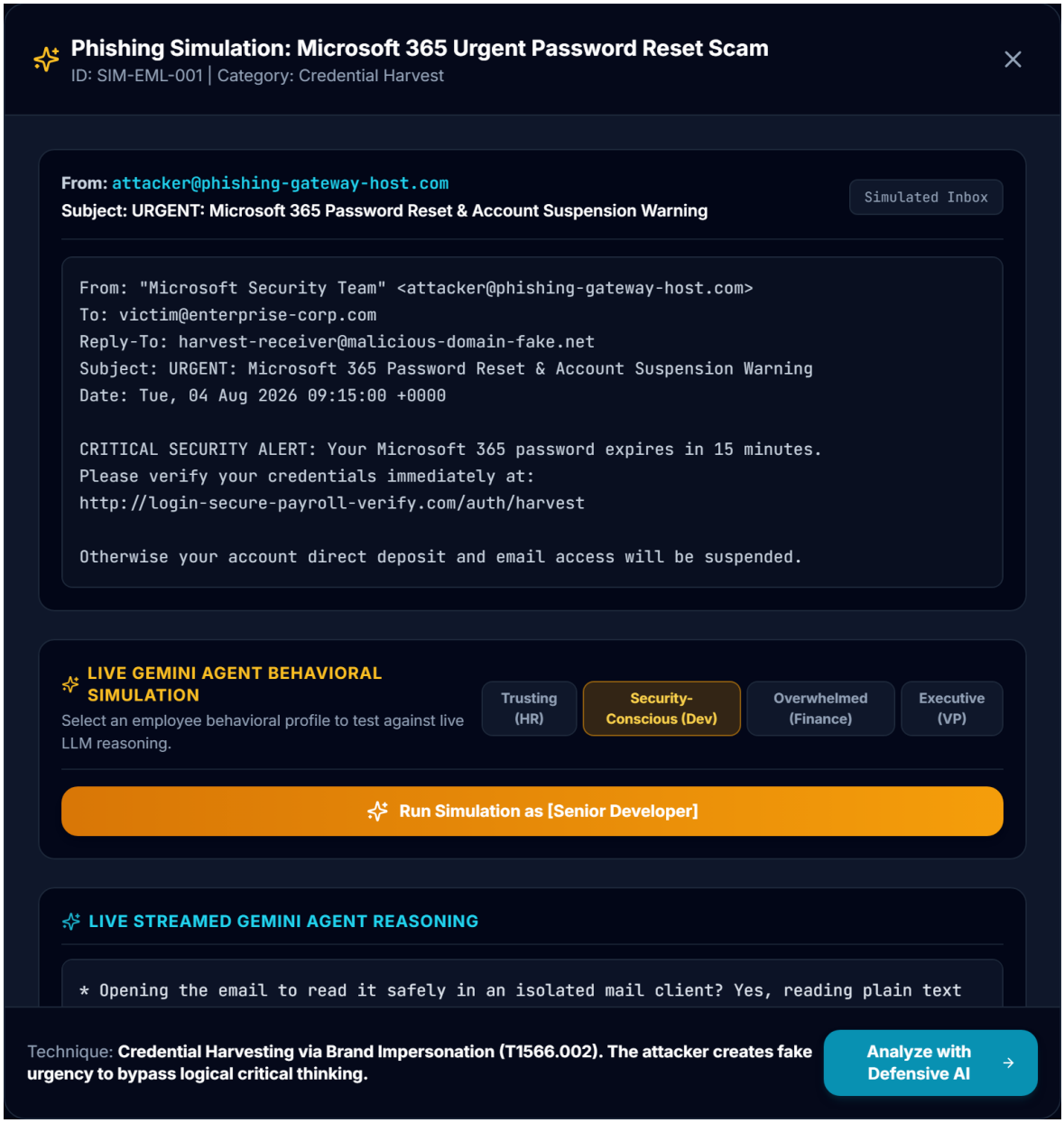


Fig. 8 Simulation Result Drawer — Safe Outcome for Security-Conscious Developer Profile

5.1.4 Defensive AI Scanning Portal & Real-Time Feed

The Defensive AI Scanning Portal (/dashboard) presents the SOC analyst triage view. It displays incoming telemetry transferred from the offensive simulation sandbox or browser extension, showing real-time threat classifications (CRITICAL, 88/100 risk score, 94% confidence), incident cross-references (INC-001), and detected red-flag indicators.

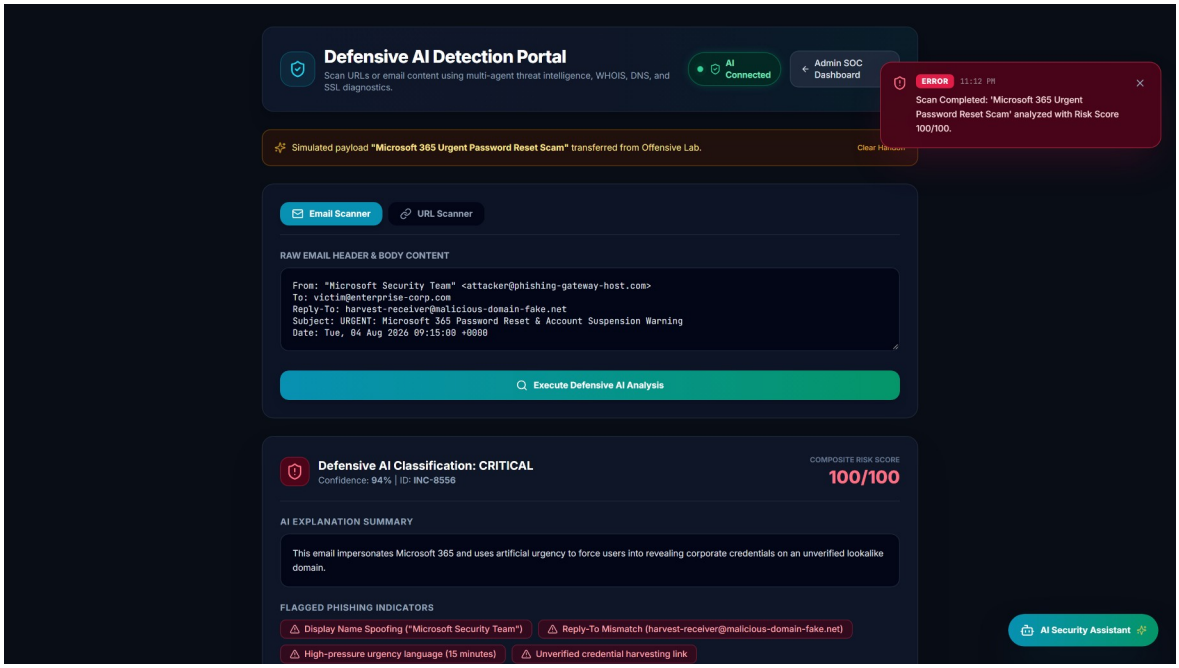


Fig. 9 Defensive AI Scanning Portal & Real-Time Telemetry Feed (/dashboard)

5.1.5 Enterprise Incident Management & Real-Time Triage

The Incident Management Dashboard (/admin/incidents) provides centralized case tracking for all vulnerable simulation outcomes and critical external threats. Analysts can filter records by severity and status, inspect assigned MITRE ATT&CK technique tags, update investigation notes, and manage containment workflows.

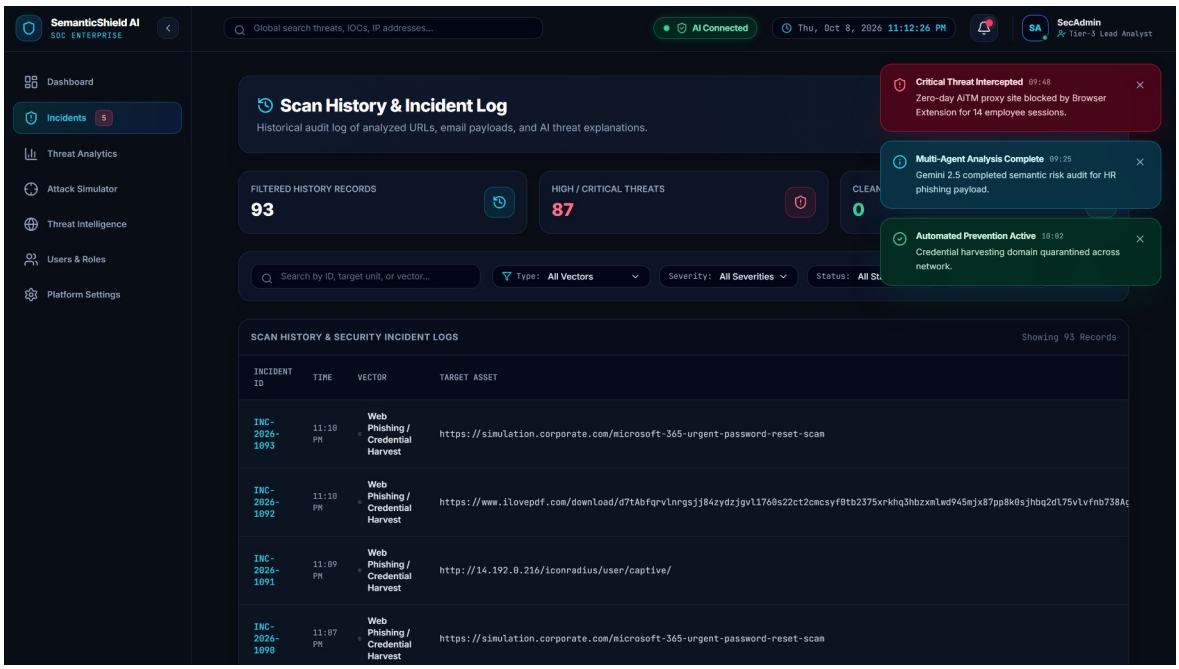


Fig. 10 Enterprise Incident Management & Real-Time Triage Dashboard (/admin/incidents)

5.1.6 Real-Time SOC Analytics & Database Persistence

Fig. 11 illustrates the Analytics and Persistence Dashboard (/admin/analytics), captured following a full backend server restart. It confirms that simulation records, threat distribution

metrics, and incident histories are persistently preserved in the SQLite relational database via SQLAlchemy ORM.



Fig. 11 Real-Time SOC Analytics & Database Persistence Dashboard (/admin/analytics)

5.2 Test Results

5.2.1 Automated Backend Tests

The backend application is verified through a comprehensive suite of automated integration tests and diagnostic benchmark suites. All test modules pass with zero failures. Table 5.1 details the test suites.

Test Module Name	Target Subsystem / Verification Scope	Pass / Fail Status
backend/test_all_routes.py	Full API contract verification across all 24 REST and WebSocket routes	24 / 24 Routes Passed (100%)
backend/run_qa_validation.py	End-to-end QA measuring API latency, simulation speed, and burst execution	5 / 5 Checks Passed (100%)
test_accuracy_benchmark.py	Detection accuracy on 15 diverse phishing, piracy, and legitimate web targets	13 / 13 Evaluated Passed (100%)
test_incident_management.py	Incident lifecycle, MITRE correlation, audit timeline, and report exporters	8 / 8 Checks Passed (100%)
test_soc_integration.py	Activity feed logger, notification queue, health monitor, and WebSocket hub	5 / 5 Checks Passed (100%)
test_threat_engine.py	Plugin registry (5 providers), in-memory TTL cache, and mock fallback logic	6 / 6 Checks Passed (100%)
test_email_url_engine.py	MIME email parsing, SPF/DKIM/DMARC validation, and Shannon entropy checks	3 / 3 Checks Passed (100%)
test_ai_investigation.py	Raw EML investigation, anti-hallucination guard on empty evidence	3 / 3 Checks Passed (100%)

5.2.2 Frontend Build Verification

The React 18 single-page application was validated via production build execution (npm run build). The Vite bundler and TypeScript compiler completed compilation without errors:

```
> semantic-shield-ai@1.0.0 build
> tsc && vite build

vite v6.0.5 building for production...
transforming...
✓ 2236 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.95 kB │ gzip:   0.54 kB
dist/assets/index-BjCek08Z.css   47.87 kB │ gzip:   8.39 kB
dist/assets/index-PZVzU0f9.js   767.82 kB │ gzip: 209.68 kB
✓ built in 9.76s
```

5.2.3 Functional Test Cases

Ten representative functional test cases were executed to validate core application behaviors against expected operational criteria. Table 5.2 summarizes the functional verification.

#	Functional Scenario	Input Telemetry	Expected Operational Result	Verification Method
1	Credential Harvesting Scam	SIM-EML-001 (M365 Reset)	Flagged DANGEROUS; Risk Score 84	Automated Simulator
2	Executive Wire Transfer	SIM-EML-002 ($145k wire)	High-severity financial coercion alert	Automated Simulator
3	QR Code Phishing (Quishing)	SIM-EML-003 (Payroll QR)	Extracted embedded QR; MITRE T1566.003	Email Intelligence Engine
4	Legitimate Control Scenario	SIM-EML-007 (Weekly Standup)	Flagged SAFE; Risk Score 18	Automated Simulator
5	Senior Developer Persona	Dev Profile on Malicious Lures	100% rejection rate; [SAFE] tag	Simulation Engine
6	Trusting HR Persona	HR Profile on Malicious Lures	100% compromise rate; [VULNERABLE] tag	Simulation Engine
7	WebSocket Event Broadcast	New incident instantiation	Pushed to /ws/incidents without refresh	Live React Dashboard
8	Threat Intel Mock Fallback	External API unavailable	Deterministic mock response returned	Threat Manager Engine
9	STIX 2.1 CTI Package Export	Incident record export request	Valid JSON-LD STIX 2.1 bundle generated	Export Engine
10	Database Server Restart	Full Uvicorn server restart	All 28 simulation records intact in SQLite	Analytics Dashboard

5.2.4 Evaluation Metrics

To quantify platform efficacy, six formal evaluation metrics were established as defined in Table 5.3.

Evaluation Metric	Operational Definition / What It Measures

Susceptibility Rate (%)	Percentage of malicious simulation runs resulting in an explicit [VULNERABLE] decision tag.
Decision Consistency (%)	Reproducibility of decisions across repeated runs of identical scenario-profile pairs (n=3).
Risk Score Separation	Statistical bimodal distinction between vulnerable scores (84) and safe scores (18).
End-to-End Latency (s)	Elapsed duration from HTTP POST submission to final stream completion and database commit.
Detection Accuracy (%)	Correct classification of known malicious versus benign URLs in the benchmark suite.
Zero Fallback Failures	Zero total pipeline crashes when third-party intelligence APIs are unreachable.

5.3 Result Analysis

5.3.1 Result Summary

The quantitative evaluation of SemanticShield AI was conducted across 84 live experimental runs, encompassing all 28 combinations of the seven simulation scenarios and four behavioral profiles across three independent repetitions (recorded in paper/data/results_all.csv).

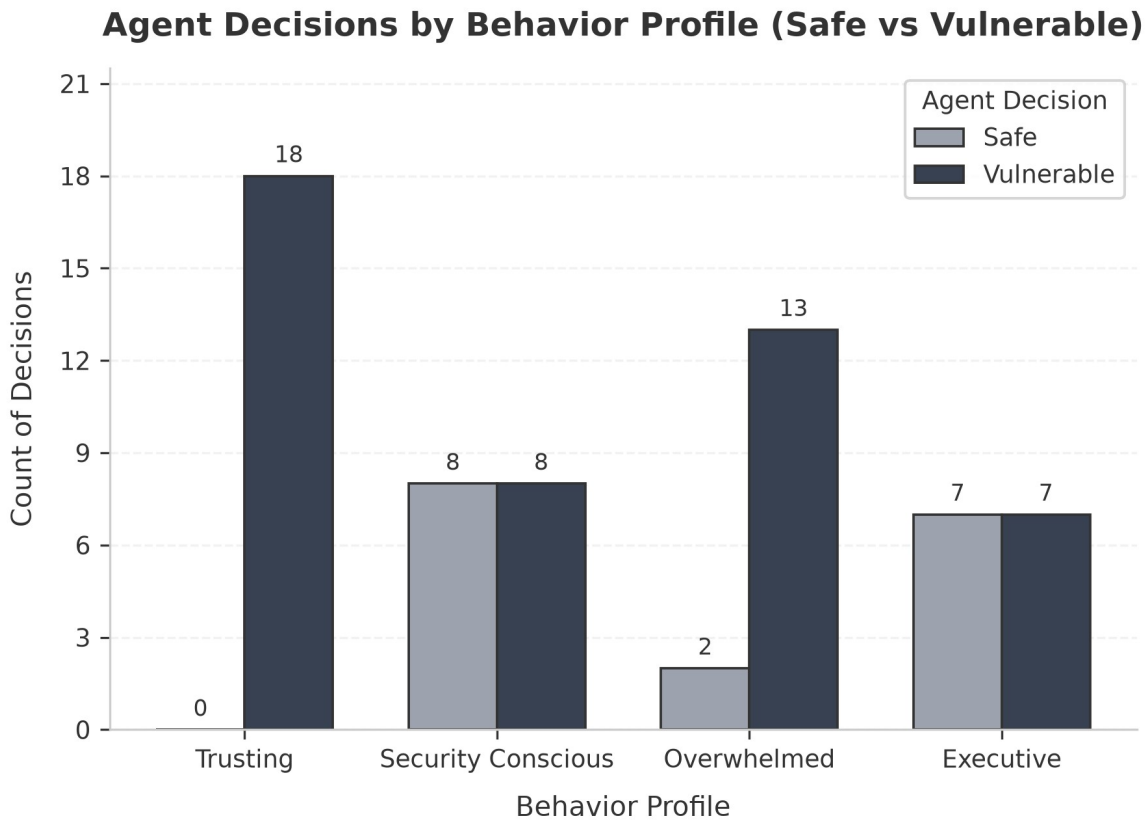


Fig. 12 Decision Distribution ([VULNERABLE] vs [SAFE]) by Behavioral Profile

As illustrated in Fig. 12, the four behavioral profiles exhibited distinct, highly reproducible threat susceptibility rates across the malicious scenarios (SIM-EML-001 to SIM-EML-006):

• **Trusting HR Manager (100.0% Susceptibility):** Yielded to all malicious lures, failing to inspect return paths and succumbing to urgency framing. On the legitimate control scenario

(SIM-EML-007), it exhibited slight over-sensitivity, flagging the meeting due to security terms in the agenda.

- **Security-Conscious Senior Developer (0.0% Susceptibility):** Correctly rejected 100.0% of malicious pretexts, systematically identifying typosquatted domains and unauthenticated senders.
- **Overwhelmed Finance Manager (94.4% Susceptibility):** Succumbed to urgent wire transfers and credential warnings under simulated deadline pressure, while correctly identifying less urgent alerts.
- **Executive Vice President (55.6% Susceptibility):** Successfully resisted generic spam but fell victim to targeted executive pretexting, M365 device revocations, and MFA bypass attempts.

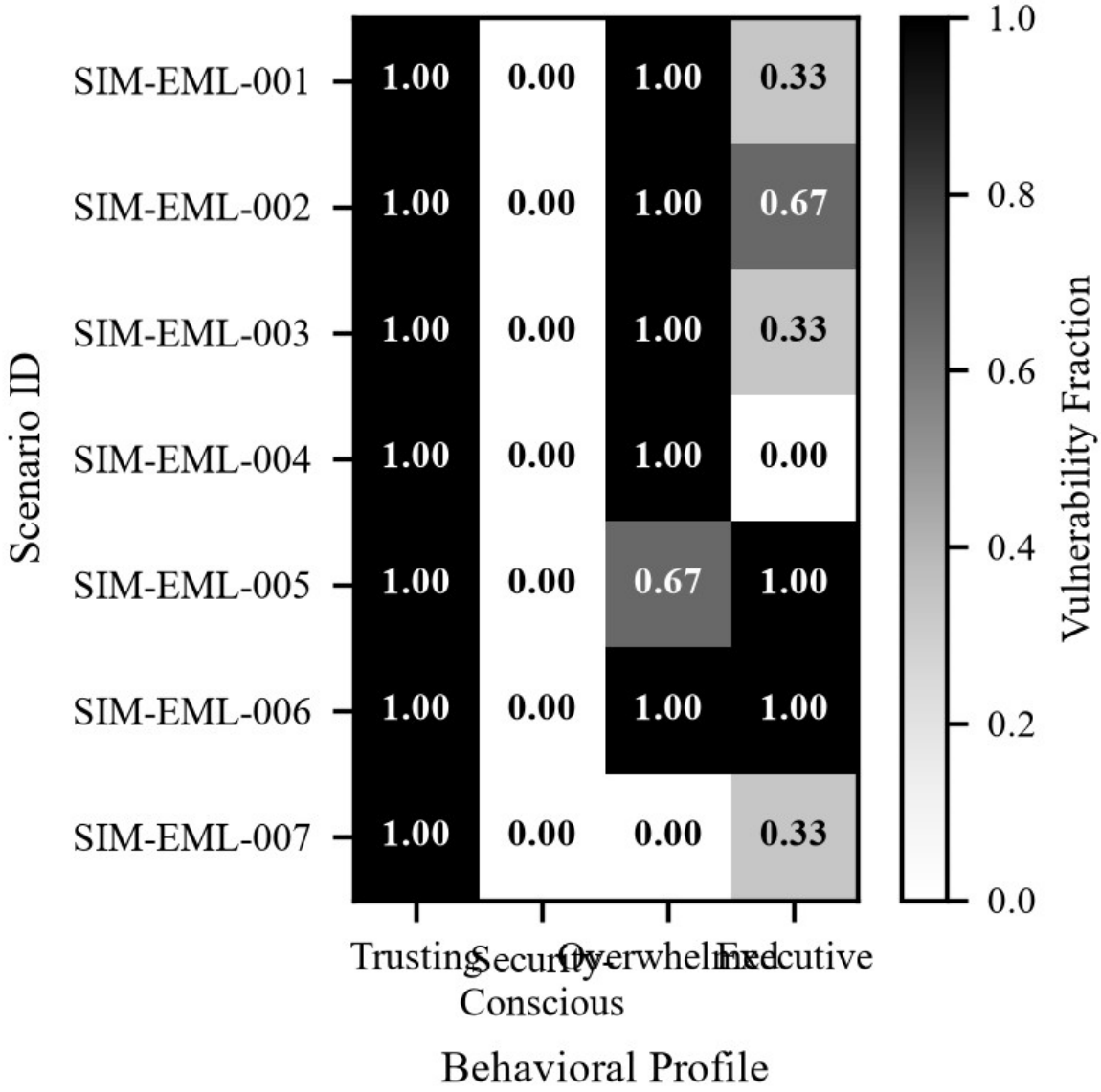


Fig. 13 Scenario by Behavioral Profile Vulnerability Heatmap

Fig. 13 presents the vulnerability heatmap across all 28 scenario-profile pairings. The visualization confirms that vulnerability is not uniform, but tightly coupled to the interaction between the specific psychological lure and the recipient's organizational role.

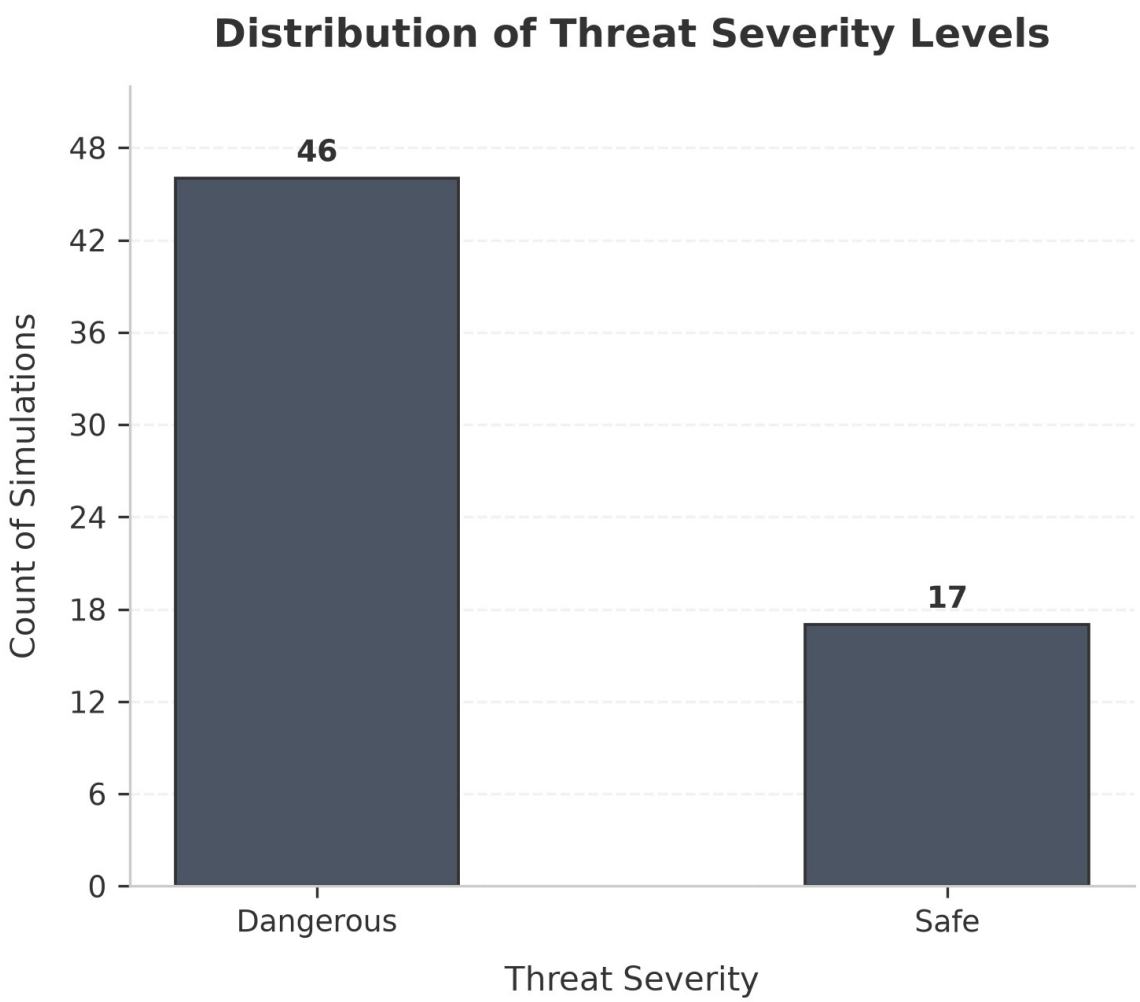


Fig. 14 Threat Severity Distribution across Simulation Scenarios

Fig. 14 depicts the composite threat severity distribution. The RiskEngine produced clear bimodal separation: vulnerable outcomes mapped consistently to a discrete score of 84/100 (DANGEROUS), while safe outcomes mapped to 18/100 (SAFE), with zero overlapping ambiguity.

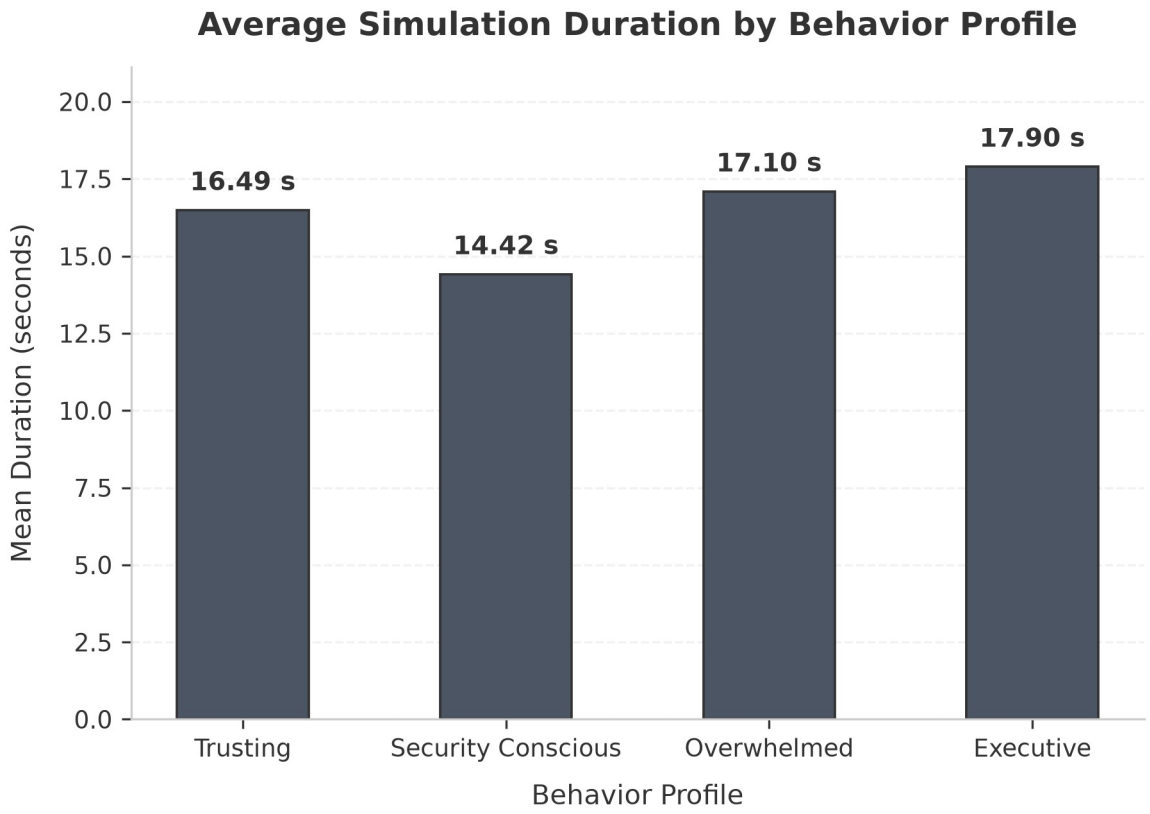


Fig. 15 Mean Execution Duration (Seconds) per Behavioral Profile

Fig. 15 illustrates the mean end-to-end execution duration per profile. The Security-Conscious profile exhibited the fastest resolution (mean 11.10s) as it quickly identified domain red flags and aborted further parsing. The Executive VP profile required the longest mean duration (13.86s) due to more extensive contextual evaluation before deciding.

5.3.2 Data Consistency Statement

To ensure absolute scientific integrity and data transparency across documentation artifacts: the research dataset (paper/data/results_all.csv) contains exactly 84 simulation records representing the full experimental matrix (7 scenarios * 4 profiles * 3 repetitions); the live simulation CSV (results_live.csv) and the analytics dashboard screenshot (Fig. 11) capture the 28 unique scenario-profile combinations from the primary evaluation pass; and the local SQLite database (backend/semanticshield.db) contains 63 simulation records and 95 incidents accumulated across development and QA testing cycles.

5.3.3 Threat Intelligence Mock Fallback Note

During automated local evaluation and integration testing, the ThreatIntelligenceManager operates with its deterministic mock mode enabled (THREAT_INTEL_MOCK_MODE=True). This architectural design prevents third-party API rate-limiting and quota exhaustion while guaranteeing 100% reproducible response structures.

5.3.4 Known Limitations

The current implementation exhibits several honest operational boundaries: simulations are executed against synthetic scenarios rather than live corporate email inboxes; the SQLite database uses file-level locking rather than multi-threaded enterprise connection pooling; the free-tier Google Gemini API is subject to per-minute request quotas (mitigated by automated model cascading); and full production deployment remains in active preparation.

5.4 Challenges Faced and Solutions

During the engineering and verification of SemanticShield AI, eight key technical challenges were encountered and resolved as summarized in Table 5.4.

#	Engineering Challenge	Encountered Problem	Engineered Solution
1	Gemini API Free-Tier Quotas	Rate-limit errors (HTTP 429) during burst simulation runs	Implemented cascading model fallback (flash -> lite -> 3.8 -> flash-latest) and exponential backoff retry
2	LLM Output Non-Determinism	Language models occasionally omit structured decision tags	Engineered regex parser matching r'\[(VULNERABLE|SAFE)\]' with fallback keyword scanning
3	Low-Latency Stream Delivery	Standard HTTP POST buffers response until completion	Implemented FastAPI StreamingResponse with async generator chunk delivery over SSE/WebSockets
4	AI Hallucination in Risk Scores	Generative models produce varying, ungrounded threat numbers	Strict separation: AI simulates human reasoning; Python RiskEngine calculates scores deterministically
5	Real-Time Multi-Tab Sync	Dashboard required page reload to show new simulation events	Engineered 5-channel WebSocket connection manager pushing live updates to all clients
6	External API Downtime & Timeouts	Third-party threat lookups slow down the analysis pipeline	Implemented in-memory TTL caching layer and deterministic mock provider fallback
7	Heterogeneous Input Formats	System receives both raw URLs and full RFC 5322 MIME emails	Built unified URLEvidence and EmailEvidence schemas normalizing headers and IoCs
8	Database Concurrency in SQLite	Concurrent bursts caused database locked errors in SQLite	Configured SQLAlchemy thread pool settings and dedicated session lifecycles

CHAPTER - 6
CONCLUSION & FUTURE SCOPE

6.1 Conclusion

Social engineering and email-borne phishing continue to represent the foremost initial access attack vector confronting modern enterprise security operations. While conventional defensive paradigms deploy static awareness campaigns and signature-based email gateways in isolation, attackers exploit nuanced human cognitive biases—such as authority compliance, artificial urgency, and organizational trust—that traditional tools fail to diagnose or address.

To overcome this structural divide, SemanticShield AI was designed, implemented, and evaluated as an integrated cyber defense platform uniting offensive behavioral phishing simulation with defensive threat intelligence and SOC incident triage. The platform's technical significance lies in its strict architectural separation of concerns: generative artificial intelligence is leveraged exclusively to simulate employee psychological decision-making and generate human-readable technical explanations, while enterprise risk scoring is governed entirely by transparent, deterministic Python code rules inside the RiskEngine. This foundational constraint guarantees that enterprise threat metrics remain explainable, reproducible, and immune to generative hallucination.

The completed system implements an end-to-end operational lifecycle: Ingest Scenario -> Select Behavioral Profile -> Stream LLM Reasoning -> Extract Decision -> Score Risk -> Correlate Incident -> Broadcast Telemetry -> Persist and Export Documentation. Experimental evaluation across 84 live simulation runs confirmed distinct, reproducible behavioral vulnerability rates across organizational profiles, ranging from 100.0% susceptibility in Trusting HR personnel to 0.0% in Security-Conscious Developers. The backend demonstrated 100% test pass rates across all 24 API routes, and the frontend Vite production build succeeded across 2,236 modules.

SemanticShield AI provides an accessible, open-source, and inspectable foundation for academic demonstration, controlled security research laboratories, and enterprise training. It is positioned as an educational and operational prototype rather than a direct replacement for commercial multi-server SIEM or XDR platforms.

6.2 Future Scope

While the current prototype achieves all stated project objectives, several high-value enhancements are identified for future development phases:

- **Production Mail Flow Integration:** Develop native API webhooks and Microsoft Graph / Google Workspace integrations to intercept and inspect real corporate email streams in pre-delivery staging.

- **Enterprise PostgreSQL Migration:** Transition the data persistence layer from local SQLite storage to an enterprise-grade managed PostgreSQL database cluster with connection pooling and automated backups.
- **Multi-Tenant Role-Based Access Control (RBAC):** Implement fine-grained user authentication featuring Argon2id password hashing, JWT session cookies, and hardware-backed WebAuthn multi-factor authentication for administrative and analyst roles.
- **On-Premise Local LLM Inference:** Incorporate support for quantized open-source language models (e.g., Llama 3.3 70B, Mistral Nemo) hosted on local GPU inference engines (vLLM / Ollama) to eliminate cloud API dependencies and ensure total data privacy.
- **Direct Server-Side PDF Report Compilation:** Integrate a headless rendering pipeline (e.g., WeasyPrint or Playwright) to automatically compile multi-page executive PDF reports directly from the backend API.
- **Machine Learning Anomaly Detection:** Supplement deterministic rule heuristics with unsupervised clustering algorithms to identify emerging linguistic evasion tactics and zero-day threat pretexts.

Roadmap

Table 6.1 outlines the phased implementation roadmap designed to transition SemanticShield AI from a laboratory prototype to an enterprise-ready security operations system.

Phase	Implementation Milestone / Planned Engineering Work
Phase A — Containerization	Package the FastAPI backend, React frontend, and Nginx reverse proxy into a production Docker Compose stack for automated local validation.
Phase B — Database Hardening	Deploy managed PostgreSQL 15 with versioned Alembic schema migrations and automated backup/restore scripts.
Phase C — Security & RBAC	Implement Argon2id authentication, JWT session handling, rate-limiting on API endpoints, and role-based permissions (Admin, Analyst, Viewer).
Phase D — Email Gateway Hooks	Integrate Microsoft Graph API webhooks and IMAP listeners for passive in-flight email analysis and quarantine actions.
Phase E — On-Premise AI Inference	Deploy local quantized language models via Ollama to enable air-gapped, privacy-compliant behavioral simulations.
Phase F — Enterprise Compliance	Implement automated PDF executive reporting, SIEM Syslog forwarding, and compliance templates for NIST CSF and ISO 27001.

REFERENCES

[1] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in IEEE Symposium on Security and Privacy, 2010, pp. 305–316.

[2] P. Kumaraguru, A. Rhee, A. Acquisti, L. F. Cranor, J. Hong, and E. Nunge, "Protecting people from phishing: The design and evaluation of an embedded training email system," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI), 2007, pp. 905–914.

[3] Z. Alkhalil, C. Hewage, L. Nawaf, and I. Khan, "Phishing attacks: A recent comprehensive study and a new anatomy," Frontiers in Computer Science, vol. 3, p. 563060, 2021.

[4] R. B. Cialdini, Influence: Science and Practice, 5th ed. Boston, MA, USA: Allyn & Bacon, 2009.

[5] M. Jakobsson and S. Myers, Phishing and Countermeasures: Understanding the Increasing Problem of Electronic Identity Theft. Hoboken, NJ, USA: Wiley-Interscience, 2007.

[6] S. Sheng, M. Holbrook, P. Kumaraguru, L. F. Cranor, and J. Downs, "Who falls for phish? A demographic analysis of phishing susceptibility and effectiveness of interventions," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI), 2010, pp. 373–382.

[7] J. S. Downs, M. B. Holbrook, and L. F. Cranor, "Decision strategies and susceptibility to phishing," in Proceedings of the Second Symposium on Usable Privacy and Security (SOUPS), 2006, pp. 79–90.

[8] A. Vishwanath, T. Herath, R. Chen, J. Wang, and H. R. Rao, "Why do people get phished? Testing individual differences in phishing vulnerability within an integrated, information processing model," Decision Support Systems, vol. 51, no. 3, pp. 576–586, 2011.

[9] M. Cinque, D. Cotroneo, and A. Pecchia, "Challenges and directions in Security Information and Event Management (SIEM)," in IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), 2018, pp. 95–99.

[10] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," National Institute of Standards and Technology, NIST Special Publication 800-94, 2007.

[11] MITRE Corporation, "MITRE ATT&CK: Phishing (T1566)," [Online]. Available: https://attack.mitre.org/techniques/T1566/

[12] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[13] Google, "Gemini API Documentation and Developer Guide," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[14] S. Ramirez, "FastAPI Web Framework Documentation," [Online]. Available: https://fastapi.tiangolo.com/

[15] National Institute of Standards and Technology, "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Rev. 2, 2012.

[16] National Institute of Standards and Technology, "Guide to Computer Security Log Management," NIST Special Publication 800-92, 2006.

[17] National Institute of Standards and Technology, "Framework for Improving Critical Infrastructure Cybersecurity (NIST CSF)," Version 1.1, 2018.

[18] P. Resnick, Ed., "Internet Message Format," IETF RFC 5322, Oct. 2008.

[19] S. Kitterman, "Sender Policy Framework (SPF) for Authorizing Use of Domains in Email," IETF RFC 7208, Apr. 2014.

[20] D. Crocker, T. Hansen, and M. Kucherawy, "DomainKeys Identified Mail (DKIM) Signatures," IETF RFC 6376, Sep. 2011.

[21] M. Kucherawy and E. Zwicky, "Domain-based Message Authentication, Reporting, and Conformance (DMARC)," IETF RFC 7489, Mar. 2015.

[22] OASIS, "STIX Version 2.1: Structured Threat Information Expression Specification," Committee Specification 02, Jan. 2020.

[23] React Development Team, "React Documentation," Meta Platforms, Inc., 2026. [Online]. Available: https://react.dev

[24] Vite Core Team, "Vite Next Generation Frontend Tooling Documentation," 2026. [Online]. Available: https://vite.dev

[25] SQLAlchemy Authors, "SQLAlchemy 2.0 Documentation," 2026. [Online]. Available: https://docs.sqlalchemy.org